A PROJECT REPORT ON

MedIntel: A Multi-Agent AI Framework for Clinical Decision Support

SUBMITTED TO

THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

FOR THE PARTIAL FULFILLMENT OF THE DEGREE OF

BACHELOR OF ENGINEERING

(Computer Engineering)

BY

Siddhesh Kishor Suryawanshi	Exam No:
Shantanu Avinash Patil	Exam No:
Omkar Abaji Musale	Exam No:
Tanmay Shriniwas Bagul	Exam No:

Under The Guidance of

Prof. G. P. Mohole

ISO 21001:2018 Certified | NAAC Accredited

DEPARTMENT OF COMPUTER ENGINEERING

JAWAHAR EDUCATION SOCIETY'S INSTITUTE OF TECHNOLOGY, MANAGEMENT AND RESEARCH, NASHIK

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

2026–2027 (Sem I)

ISO 21001:2018 Certified | NAAC Accredited

Jawahar Education Society's,
INSTITUTE OF TECHNOLOGY,
MANAGEMENT & RESEARCH, NASHIK.
(Approved by AICTE, New Delhi, DTE, Government of Maharashtra, Affiliated to Savitribai Phule Pune University)

CERTIFICATE

This is to certify that the Project Entitled

MedIntel: A Multi-Agent AI Framework for Clinical Decision Support

Submitted by

Siddhesh Kishor Suryawanshi	Exam No:
Shantanu Avinash Patil	Exam No:
Omkar Abaji Musale	Exam No:
Tanmay Shriniwas Bagul	Exam No:

is a bonafide work carried out by them under the supervision of Prof. G. P. Mohole and is approved for Phase I as partial fulfilment of the requirements of Savitribai Phule Pune University, Pune, for the award of the degree of Bachelor of Engineering (Computer Engineering).

Prof. G. P. Mohole	**Prof. G. P. Mohole**	**Dr. M. V. Bhatkar**
Project Guide	H.O.D	Principal
Department of	Department of	JESITMR,
Computer Engineering	Computer Engineering	Nashik

Signature of Internal Examiner **Signature of External Examiner**

PROJECT APPROVAL SHEET

A

Project

on

MedIntel: A Multi-Agent AI Framework for Clinical Decision Support

is successfully completed by

Siddhesh Kishor Suryawanshi	Exam No:
Shantanu Avinash Patil	Exam No:
Omkar Abaji Musale	Exam No:
Tanmay Shriniwas Bagul	Exam No:

at

DEPARTMENT OF COMPUTER ENGINEERING

JIT
ISO 21001:2018 Certified | NAAC Accredited

Jawahar Education Society's,
INSTITUTE OF TECHNOLOGY, MANAGEMENT & RESEARCH, NASHIK.
(Approved by AICTE, New Delhi, DTE, Government of Maharashtra, Affiliated to Savitribai Phule Pune University)

SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

2026–2027 (Sem I)

Prof. G. P. Mohole	**Prof. G. P. Mohole**
Project Guide	H.O.D
Department of Computer Engineering	Department of Computer Engineering

ACKNOWLEDGEMENT

We take this opportunity to thank everyone who guided and encouraged us while we worked on our project, “MedIntel: A Multi-Agent AI Framework for Clinical Decision Support.” Bringing machine learning, rule-based reasoning and knowledge retrieval together in one design for clinical decision support was a demanding task, and many people helped us along the way. We owe special thanks to the Department of Computer Engineering, JESITMR, for the laboratory facilities and resources it made available, and for an environment in which we could study the problem, debate ideas and work through the design. We are also grateful to the teaching and non-teaching staff of the department for their cooperation throughout the year.

We are deeply grateful to **Prof. G. P. Mohole**, who is both our Project Guide and our Head of Department. As our guide, **Prof. Mohole** mentored us patiently through every phase of the project, helping us frame the problem, shape the multi-agent design and plan a fair way to evaluate it. As Head of Department, **Prof. Mohole** made sure that we had the opportunity, the resources and the motivation to carry this stage of the work through to completion. We also thank our Principal, **Dr. M. V. Bhatkar**, for the leadership and support that made this work possible.

Finally, this work rests on the commitment of every member of our team: Siddhesh K. Suryawanshi, Shantanu A. Patil, Omkar A. Musale and Tanmay S. Bagul. Their combined effort in literature study, requirement analysis, architecture and UML design, evaluation planning and paper writing brought Project Stage I to completion. We are equally grateful to our families for their patience and steady encouragement during the long hours this project demanded. Our thanks also go to everyone who helped us, directly or indirectly, along the way.

Mr. Siddhesh Kishor Suryawanshi
Mr. Shantanu Avinash Patil
Mr. Omkar Abaji Musale
Mr. Tanmay Shriniwas Bagul

ABSTRACT

Clinical decision support systems increasingly combine machine learning, knowledge retrieval and large language models, but when one generative model produces the final recommendation, the evidence behind it becomes hard to trace or audit. Real hospital records, with missing fields, inconsistent units and free-text notes, add to the difficulty. This report presents MedIntel, a proposed multi-agent framework that treats each patient's analysis as a coordinated pipeline of components with explicit responsibilities rather than a single prediction step.

MedIntel accepts structured patient records with related event logs, validates and normalizes them, and builds one case per patient. A machine-learning agent supplies a prediction with class probabilities, a rule-based agent runs deterministic checks such as clinical thresholds, and a retrieval agent returns supporting passages with source identifiers. A language-model agent reconciles these outputs and reports any conflict, and a decision-fusion layer computes an explicit confidence score that weighs model, rule and retrieval support against disagreement among the components. Each case ends in a structured report for clinician review and a matching audit record. Cases can run in parallel in isolated contexts, and only clinician-validated outcomes approved by a Knowledge Curator are re-indexed into the knowledge store.

This report covers Project Stage I, a design-stage study, so it makes no claim of accuracy or clinical performance. It specifies the requirements, architecture, UML diagrams, data structures, reference module logic and test plan, and defines an evaluation protocol for Stage II based on progressively richer baselines, component ablation, calibration measures and human review of explanation quality. By keeping learned, rule-based and retrieved evidence separate, MedIntel aims to make patient-level clinical reasoning more traceable and easier to audit.

Keywords: *Clinical Decision Support, Multi-Agent Systems, Retrieval-Augmented Generation, Rule-Based Reasoning, Explainable AI, Healthcare AI*

TABLE OF CONTENTS

LIST OF ABBREVIATIONS		VIII
LIST OF FIGURES		X
LIST OF TABLES		XI

CHAPTER NO.	TITLE	PAGE NO.
1	**Introduction**	
1.1	Project Overview And Core Concept	12
1.2	Motivation: Making Clinical AI Traceable And Auditable	12
1.3	Problem Statement: Limitations Of Single-Model Clinical Decision Support	13
1.4	Project Aim, Objectives, And Expected Outcomes	13
1.5	Project Scope	14
1.6	Report Organization	14
2	**Literature Review**	
2.1	Overview Of AI In Clinical Decision Support	16
2.2	Critical Review Of Existing Systems And Identified Gaps	16
2.2.1	Limitations: Single-Model Reasoning And Weak Traceability	18
2.2.2	Conceptual Diagram Of Existing CDSS Approaches	18
2.2.3	The Need For Evidence Separation And Human Oversight	19
2.3	Review Of Hybrid Decision-Support Techniques	19
2.3.1	Machine Learning And Rule-Based Clinical Reasoning	20
2.3.2	Retrieval-Augmented Generation For Clinical Evidence	20
2.4	Value Of Multi-Agent Collaboration In Medical Reasoning	21
2.5	Review Of Technical Stacks For Clinical AI Systems	22
2.6	Research Gap Identification	22

3	**Software & Hardware Requirement Specification (SRS)**		
	3.1	Introduction To System Requirements	24
	3.1.1	Project Scope	24
	3.1.2	User Classes And Characteristics	24
	3.1.3	Assumptions And Technical Dependencies	25
	3.2	Functional Requirements (FRs)	26
	3.3	External Interface Requirements	27
	3.3.1	User Interfaces	27
	3.3.2	Software Interfaces	27
	3.3.3	Communication Interfaces	28
	3.4	Hardware And Software Requirements For Development And Deployment	28
	3.5	Non-Functional Requirements (NFRs)	29
4	**System Design And Architecture**		
	4.1	System Architecture And Component Overview	31
	4.1.1	High-Level Architecture Diagram	31
	4.1.2	Data Flow Diagram (DFD) Of Patient Case Processing	32
	4.2	Agent Design Details	35
	4.2.1	Agent Responsibilities And Output Contracts	35
	4.2.2	Design Of The Decision Fusion And Confidence Logic	36
	4.2.3	Multi-Patient Parallel Execution And Knowledge Update Loop	37
	4.3	Unified Modeling Language (UML) Diagrams	38
	4.3.1	Use Case Diagram For Primary Actors	38
	4.3.2	Class Diagram Of Core System Objects	39
	4.3.3	Sequence Diagram For Patient Case Analysis	40
	4.4	Data Structure Design	43
	4.4.1	Rulebase Data Structure	43
	4.4.2	JSON Schema For The Decision Report	44
5	**Implementation Details**		
	5.1	Introduction To Implementation Tools And Environment	46
	5.2	Core Pipeline And Orchestration	46
	5.2.1	Project Initialization And Data Loading	46

	5.2.2	Implementation Of The Orchestrator And Case Queue	47
5.3		Coding The Analysis Agents	48
	5.3.1	Machine-Learning Analysis Agent	48
	5.3.2	Rule-Based Validation Agent	49
5.4		Key System Implementation	50
	5.4.1	Knowledge Retrieval Agent	50
	5.4.2	Decision Fusion And Explanation Generation	50
5.5		Code Snippets And Documentation	52
	5.5.1	Critical Python Functions (Audit Logging And Report Export)	52
	5.5.2	Implementation Of The Clinician Review Interface	53
6	**Conclusion**		
6.1		Conclusion Of The Project	55
6.2		Summary Of Contribution And Uniqueness	56
6.3		Challenges Faced And Mitigation Strategies	56
References			59
Appendices			
		APPENDIX A : Base Paper	61
		APPENDIX B : IEEE Paper	63
		APPENDIX C : Plagiarism Analysis	67
		APPENDIX D : Publication	70

LIST OF ABBREVIATIONS

ABBREVIATION	FULL FORM
AI	Artificial Intelligence
CSV	Comma-Separated Values
ML	Machine Learning
LLM	Large Language Model
UML	Unified Modeling Language
SRS	Software & Hardware Requirement Specification
FRs	Functional Requirements
NFRs	Non-Functional Requirements
IEEE	Institute of Electrical and Electronics Engineers
CDSS	Clinical Decision Support System
RAG	Retrieval-Augmented Generation
ID	Identifier
GPT	Generative Pre-Trained Transformer
MACD	Multi-Agent Clinical Diagnosis
XGBoost	Extreme Gradient Boosting
FAISS	Facebook AI Similarity Search
JSON	JavaScript Object Notation
API	Application Programming Interface
REST	Representational State Transfer
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
VS Code	Visual Studio Code
LTS	Long-Term Support
RAM	Random Access Memory
SSD	Solid-State Drive
GPU	Graphics Processing Unit
VRAM	Video Random Access Memory
DFD	Data Flow Diagram
UTC	Coordinated Universal Time
TC	Test Case
BP	Blood Pressure

ROC-AUC	Area Under the Receiver Operating Characteristic Curve
EHR	Electronic Health Record
HL7	Health Level Seven
FHIR	Fast Healthcare Interoperability Resources

LIST OF FIGURES

Figure No.	Title / Description	Chapter No.
Fig 2.2.2	Conceptual Comparison Of CDSS Design Approaches	2
Fig 2.3.2	Retrieval-Augmented Generation Pipeline For Clinical Evidence	2
Fig 4.1.1	MedIntel High-Level System Architecture	4
Fig 4.1.2 (a)	Level 0 Data Flow Diagram (Context Diagram)	4
Fig 4.1.2 (b)	Level 1 Data Flow Diagram	4
Fig 4.2.3	Multi-Patient Parallel Processing And Knowledge Update Loop	4
Fig 4.3.1	Use Case Diagram For Primary Actors	4
Fig 4.3.2	Class Diagram Of Core System Objects	4
Fig 4.3.3 (a)	Sequence Diagram For Patient Case Analysis	4
Fig 4.3.3 (b)	Sequence Diagram For Analyzing One Case	4

LIST OF TABLES

Table No.	Title / Description	Chapter No.
Table 2.2	Literature Review And Gap Identification	2
Table 3.1.2	User Classes And Characteristics	3
Table 3.2	Functional Requirements (FRs)	3
Table 3.4	Hardware And Software Requirements For Development And Deployment	3
Table 3.5	Non-Functional Requirements (NFRs)	3
Table 4.2.1	Agent Responsibilities And Output Contracts	4
Table 6.1	Stage I Deliverables In Numbers	6
Table 6.2	Contributions Of MedIntel In Numbers	6
Table 6.3	Challenges Faced And Mitigation Strategies	6
Table A.1	Base Paper Details	A

CHAPTER 01

INTRODUCTION

This chapter introduces MedIntel, a proposed multi-agent framework for clinical decision support. It explains the core concept of the project, the motivation for making clinical artificial intelligence (AI) traceable and auditable, and the limitations of decision support that depends on a single model. It then states the aim, objectives and expected outcomes, defines the scope of Project Stage I, and outlines the chapters that follow.

1.1 PROJECT OVERVIEW AND CORE CONCEPT :

MedIntel is a proposed framework for patient-level clinical decision support, planned as a medical log analysis system. It accepts structured patient records in CSV or tabular form, together with related event logs and short text context, and produces a structured recommendation that a clinician reviews before any action is taken. Its core concept is that each kind of evidence should be handled by the component best suited to it: a machine learning (ML) model for statistical patterns, a rule engine for explicit thresholds and policies, retrieval for contextual evidence, and a large language model (LLM) for synthesis and explanation.

A patient case moves through a fixed sequence of stages. Input ingestion validates and normalizes the record, after which the Machine Learning Analyzer, Rule-Based Validation and Knowledge Retrieval produce a prediction, a set of rule results and source-linked guideline passages. The Clinical Context Agent reconciles these outputs and reports any conflict, Decision Fusion & Confidence Assessment combines them through an explicit confidence function, and an explanation and audit stage writes the report for clinician review. Several cases run in parallel in isolated contexts, and the knowledge store grows only through outcomes that a clinician has validated and the Knowledge Curator has approved.

1.2 MOTIVATION: MAKING CLINICAL AI TRACEABLE AND AUDITABLE :

Hospital data rarely arrives in a clean form. One record may hold missing fields, inconsistent units, repeated observations, free-text notes, and event logs that describe the same episode from different angles. A decision-support tool working on such data must decide which evidence may influence a prediction, and it must explain the outcome in a

form a clinician can check. Language models make such tools more capable, yet reviews of LLM-based clinical systems report calibration, validation and transparency as recurring weaknesses [7], and the medical literature stresses risks of factuality and context dependence [11]. MedIntel is motivated by the need for recommendations that carry their own record (the model output, the rules that fired, the sources retrieved and the versions in use) so that a clinician or auditor can later reconstruct how each one was reached.

1.3 PROBLEM STATEMENT: LIMITATIONS OF SINGLE-MODEL CLINICAL DECISION SUPPORT :

Systems that rely mainly on free-form LLM inference make one generative model responsible for reading the case, recalling medical knowledge and writing the final recommendation. The result cannot be traced to the feature, rule or source behind it, explicit thresholds are not guaranteed to be applied the same way on every run, and missing evidence may be filled in from model memory instead of being reported. Adding more LLM agents brings useful specialization [1], [2], but it also makes the origin of a recommendation harder to locate.

The problem addressed by this project is therefore to design an implementable, modular pipeline for patient-level decision support that accepts imperfect patient data, keeps learned, rule-based and retrieved evidence separate, combines that evidence through an explicit and testable confidence function, and presents a traceable report for clinician review, while processing several patient records in parallel without mixing their contexts. The formal statement of the problem is given in Chapter 4.

1.4 PROJECT AIM, OBJECTIVES, AND EXPECTED OUTCOMES :

- **Aim :** MedIntel is designed to provide patient-level clinical decision support in which every recommendation is traceable to the evidence that produced it, carries an explicit confidence score, and reaches a clinician as a structured report for review instead of acting as an autonomous diagnosis.
- **Objectives :**
  - **Modular Architecture (Stage I) :** Specify the requirements and design separate ML, rule, retrieval and synthesis agents with defined output contracts, so that any one of them can be replaced independently.

- **Traceable Reporting Design (Stage I) :** Define the patient case, decision report and audit record so that each recommendation links back to its input features, triggered rules, model output and retrieved sources.
- **Parallel Prototype (Stage II) :** Implement the designed modules as a working prototype that validates incoming records and analyzes several patient cases concurrently, each in an isolated context.
- **Staged Evaluation (Stage II) :** Compare the ML model alone, ML with rules, ML with rules and retrieval, and the full fusion pipeline, together with ablation, calibration and reliability checks.
- **Clinician Review Loop (Stage II) :** Provide a dashboard for accepting, rejecting or annotating each report, and pass only clinician-validated outcomes to the Knowledge Curator for controlled re-indexing.

❖ **Expected Outcomes :** Stage I is expected to deliver a complete design: requirements, architecture, Unified Modeling Language (UML) models, data structures, module logic and a test plan. Stage II is expected to deliver a working prototype that produces evidence-linked, confidence-scored reports with audit records, along with an evaluation of what each component contributes. No accuracy claim is made before that evaluation.

1.5 PROJECT SCOPE :

This report covers Project Stage I (Semester 7), which is limited to analysis and design: the literature review, requirement specification, architecture, data flow and UML diagrams, agent contracts, confidence logic, data structures, the reference implementation of the core module logic, and the test plan. Building and evaluating the prototype belong to Project Stage II (Semester 8), so every performance figure in this report is a target rather than a measurement. The planned prototype handles de-identified structured patient records with event logs and short text context, drawn from a documented public clinical dataset, the datasets available from the base papers [1], [2], and sample data prepared by the team. Autonomous diagnosis, real-time bedside monitoring, medical imaging and live integration with hospital information systems remain outside this scope; the last is left as future work beyond Stage II.

1.6 REPORT ORGANIZATION :

The remainder of this report follows the structure of a design-stage project report. Chapters 1 to 4 define the problem, review the literature, specify the requirements and present the design. Chapter 5 describes the planned implementation, and Chapter 6 concludes the report

with the Stage I results in numbers, the contributions and the challenges faced. The chapters are organized as follows:

- **Chapter 1 :** Introduction, covering the overview, motivation, problem statement, objectives and scope.
- **Chapter 2 :** Literature Review, comparing twelve published works and identifying the research gap.
- **Chapter 3 :** Software & Hardware Requirement Specification (SRS), including FR1 to FR9 and NFR1 to NFR7.
- **Chapter 4 :** System Design And Architecture, with the architecture, data flow and UML diagrams, agent contracts and data structures.
- **Chapter 5 :** Implementation Details, giving the tools and the reference implementation of each module.
- **Chapter 6 :** Conclusion, summarizing the Stage I work in numbers, the contributions, and the challenges faced with their mitigation.
- **References And Appendices :** The twelve cited works, the base paper, the IEEE paper, the plagiarism analysis and the publication record.

SUMMARY :

Chapter 1 introduced MedIntel, a proposed multi-agent framework that analyzes patient records through separate ML, rule-based, retrieval and synthesis components and presents a traceable, confidence-scored report for clinician review. It explained why recommendations from a single generative model are hard to audit, framed the problem as the design of a modular, evidence-separated pipeline, and stated the aim, five objectives across Stage I and Stage II, and the expected outcomes. The present scope was limited to design, and the organization of the report was outlined.

CHAPTER 02

LITERATURE REVIEW

This chapter reviews the published work behind the design of MedIntel. It outlines the role of artificial intelligence (AI) in clinical decision support, compares twelve recent studies, and discusses the limitations they share. It then reviews hybrid techniques that combine machine learning (ML) with rules or retrieval, the value of multi-agent collaboration, and the technical stacks used for such systems, and it closes by identifying the research gap that MedIntel addresses.

2.1 OVERVIEW OF AI IN CLINICAL DECISION SUPPORT :

Clinical decision support systems (CDSSs) help clinicians interpret patient data through alerts, risk estimates or suggested next steps. They have traditionally been built around rules, statistical models or retrieval services that answer narrowly defined clinical questions [1]. Large language models (LLMs) widened this design space. Thirunavukarasu et al. describe broad applications of LLMs in medicine while stressing the domain-specific risks and validation requirements of clinical use [11], and later work indicates that language models can support clinical decision tasks, especially when external knowledge is retrieved at inference time [12]. Recent research falls into four complementary directions: LLM-based medical reasoning, retrieval-augmented generation (RAG), multi-agent diagnostic frameworks, and predictive or rule-based decision support. A common thread is grounding, since newer designs tie answers to explicit evidence sources instead of unrestricted model memory.

2.2 CRITICAL REVIEW OF EXISTING SYSTEMS AND IDENTIFIED GAPS :

The twelve works cited in this report were reviewed for the technique each uses, the scope of the problem it addresses, and the limitation that matters most for a traceable, patient-level system. They include multi-agent LLM frameworks, retrieval-grounded clinical systems, a retrieval-based diagnostic study and several reviews of LLM use in medicine. Table 2.2 summarizes each work in the order of its reference number.

Table 2.2 : Literature Review And Gap Identification

Ref ID	Author	Year	Technology Used	Strength	Limitation
[1]	Öğdü et al.	2025	Adaptive multi-agent LLM CDSS with biomedical RAG, web evidence and data fusion	Close to MedIntel in modularity and confidence-aware processing.	With many reasoning components, it is harder to tell which one produced a recommendation.
[2]	Tang et al.	2024	MedAgents: role-based LLM agents that discuss and reach consensus (zero-shot)	Shows the value of role specialization.	Analysis stays within LLM agents, with no dedicated ML model or deterministic rule component.
[3]	Lu et al.	2024	ClinicalRAG: multi-agent pipeline with heterogeneous knowledge retrieval	Links clinical inputs to structured and unstructured knowledge.	Tabular ML and deterministic rules do not share a common patient-level interface with retrieval.
[4]	Zuo et al.	2025	KG4Diagnosis: hierarchical LLM agents with a medical knowledge graph	Represents symptom and disease relationships explicitly.	The knowledge graph adds knowledge-engineering requirements.
[5]	Abdullahi et al.	2024	Ensemble of retrieval-based and predictive diagnostic approaches (mixed methods study)	Combining complementary methods can improve diagnostic ranking, including for less common conditions.	Focus is ranking, not an auditable patient-level pipeline.
[6]	Liu et al.	2025	Systematic review and meta-analysis of RAG in biomedicine	Reports an overall benefit of RAG over baseline LLMs.	Finds methodological weaknesses that call for stronger validation.
[7]	Delourme et al.	2024	Scoping review of LLMs used as CDSS	Identifies calibration, validation and transparency as recurring weaknesses.	Reviews existing work rather than proposing a system.
[8]	Wang et al.	2024	GPT-based LLM with retrieval-augmented generation	Argues that retrieval can make clinical responses more specific to current medical information.	A discussion of future potential rather than an evaluated system.
[9]	Kresevic et al.	2024	RAG framework with structured guideline retrieval and prompt construction	Can improve the accuracy of guideline interpretation.	Shown for one focused task (hepatology guidelines).

Ref ID	Author	Year	Technology Used	Strength	Limitation
[10]	Li et al.	2025	MACD: multi-agent LLM diagnosis with self-learned knowledge	Shows the potential of iterative multi-agent learning.	Makes the governance of self-learned knowledge more visible, without fully resolving it.
[11]	Thirunavukarasu et al.	2023	Review of large language models in medicine	Describes broad applications of LLMs in medicine.	Stresses risks of factuality and context dependence and the need for clinical validation.
[12]	Patel et al.	2026	Systematic review of fine-tuning, RAG and hybrid adaptation of language models	Language models can support clinical decisions, especially with retrieval at inference time.	Calibration, validation and transparency remain weak.

2.2.1 LIMITATIONS: SINGLE-MODEL REASONING AND WEAK TRACEABILITY :

The review points to two related limitations. The first is single-model reasoning: when the final recommendation comes from one generative step, it inherits the weaknesses that reviews of clinical LLM systems keep reporting, namely calibration, validation and transparency [7], [12]. The second is weak traceability. Multi-agent frameworks spread reasoning across several agents [1], [2], [4], but the more reasoning components a system contains, the harder it becomes to identify which one produced a particular recommendation, and errors in chained LLM calls can propagate from one agent to the next. In both cases the output is not linked to the specific feature, rule or source behind it.

2.2.2 CONCEPTUAL DIAGRAM OF EXISTING CDSS APPROACHES :

Fig 2.2.2 compares three ways of building a CDSS at a conceptual level. In the single-model design (a), patient data passes through one model or LLM whose reasoning is opaque, and only the recommendation comes out. The RAG-based design (b) adds a retriever that searches a knowledge base by similarity, so the LLM answers from the query and the retrieved passages and can cite its sources, although no deterministic check is applied. In the evidence-separated multi-agent design (c), the Machine Learning Analyzer, Rule-Based Validation and Knowledge Retrieval produce separate evidence that Decision Fusion & Confidence Assessment combines, followed by explanation, audit and clinician review. From (a) to (c), more of the evidence behind a recommendation becomes visible, and MedIntel follows design (c).

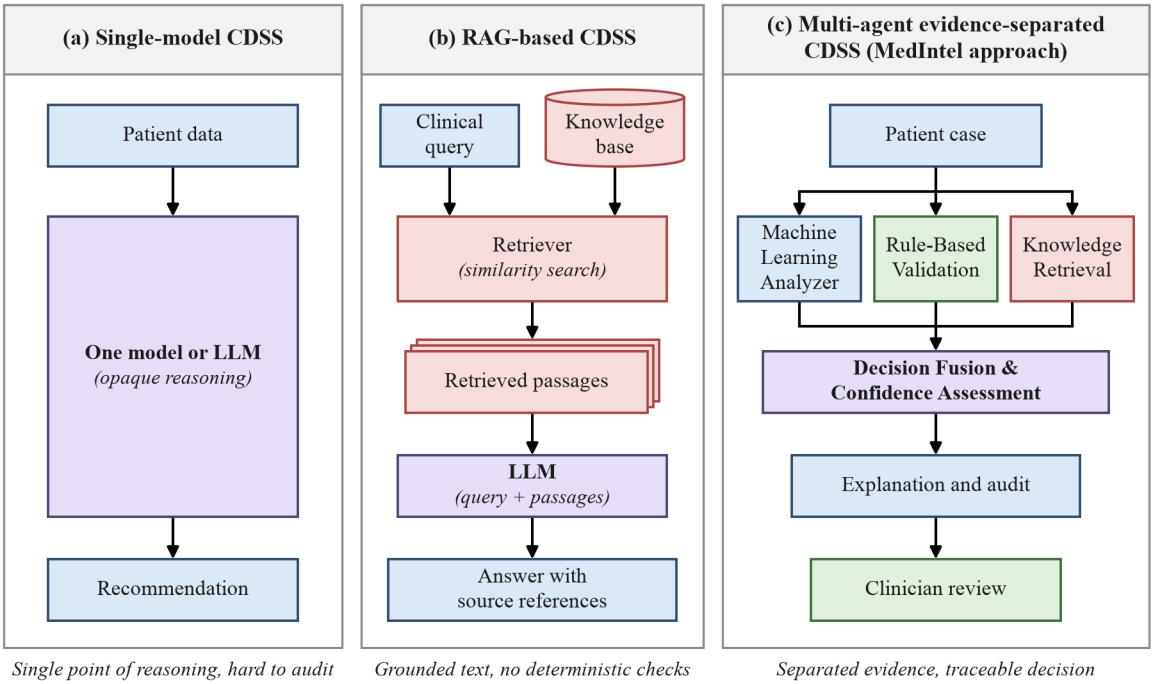


Fig 2.2.2 : Conceptual Comparison Of CDSS Design Approaches

2.2.3 THE NEED FOR EVIDENCE SEPARATION AND HUMAN OVERSIGHT :

These limitations lead to two design needs. The first is evidence separation: learned predictions, rule results and retrieved passages should remain distinct objects from the moment they are produced until fusion. Kept apart, they allow deterministic guardrails to run before any generative synthesis, let each retrieved passage keep its source identifier, and turn a disagreement between model and rules into a reported conflict instead of something smoothed over in fluent text. Separation also makes component-level testing possible, because each output can be checked against its own contract.

The second need is human oversight. The system should offer recommendations for review and never act as an autonomous diagnostic authority, and the same applies to what it learns. Work on self-learned clinical knowledge shows the potential of iterative multi-agent learning but also makes the governance problem more visible [10]: knowledge that a system writes for itself can carry its own mistakes forward into later cases. Re-indexing should therefore be limited to outcomes that a clinician has reviewed and validated.

2.3 REVIEW OF HYBRID DECISION-SUPPORT TECHNIQUES :

Hybrid decision support combines two or more reasoning techniques so that the strengths of one cover the weaknesses of another. Two hybrid families are directly relevant to MedIntel. The first pairs a learned model with explicit clinical rules, and the second, retrieval-

augmented generation, pairs a language model with an external store of medical knowledge. MedIntel brings both together inside one patient-level pipeline, so each is reviewed below.

2.3.1 MACHINE LEARNING AND RULE-BASED CLINICAL REASONING :

Machine learning suits patterns that can be learned from structured features such as age, laboratory values, vital signs and categorical attributes. Logistic regression, random forest and gradient boosting are common baselines for such tabular data. A learned model cannot, however, guarantee that a known clinical threshold is respected in every case. Rules fill that gap: a rule can flag a laboratory value above a set threshold, two findings that occur together, or a missing required field, and it gives the same answer every time it runs.

The two approaches are complementary. Rules are transparent but learn nothing from data, while learned models can capture patterns that no rule author anticipated. Abdullahi et al. found that an ensemble combining complementary retrieval and predictive approaches can improve diagnostic ranking, including for less common conditions [5], which supports letting several methods contribute to one decision. In a hybrid design, a rule may elevate, suppress or flag a candidate recommendation, but it should never override a learned prediction without recording the reason.

2.3.2 RETRIEVAL-AUGMENTED GENERATION FOR CLINICAL EVIDENCE :

Definition of Retrieval-Augmented Generation (RAG): RAG is a technique in which a language model answers a query using passages retrieved from an external knowledge collection at inference time, instead of relying only on what it learned during training. Because the passages are supplied inside the prompt, the answer can be tied to identifiable sources. In a clinical setting, the collection can hold guideline excerpts, articles and institution-approved references.

In clinical work, Wang et al. argued that retrieval can make model responses more specific to current medical information [8], and Kresevic et al. showed that structured guideline retrieval with careful prompt construction can improve the accuracy of guideline interpretation for a focused clinical task [9]. The systematic review and meta-analysis by Liu et al. reported an overall performance benefit of biomedical RAG over baseline LLMs, while identifying methodological weaknesses that call for stronger validation [6]. ClinicalRAG extends retrieval to heterogeneous structured and unstructured knowledge within a multi-agent pipeline [3].

Fig 2.3.2 shows a typical RAG pipeline for clinical evidence in two lanes. In the offline indexing lane, guideline excerpts and articles are split into passages, embedded and stored in a vector index with their source identifiers. In the online query lane, a query derived from the patient case is embedded with the same model, a top-k similarity search returns evidence passages with their source ids and scores, and a prompt built from the case and this evidence lets the LLM generate a grounded answer with citations. Retrieved text is still context rather than a diagnosis, so MedIntel treats retrieval as one scored evidence source among several.

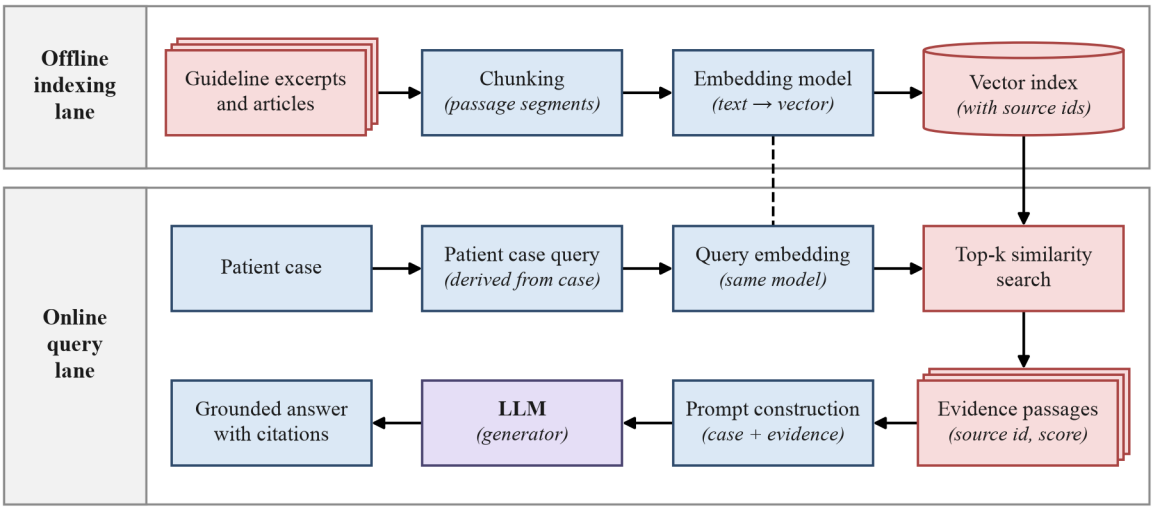


Fig 2.3.2 : Retrieval-Augmented Generation Pipeline For Clinical Evidence

2.4 VALUE OF MULTI-AGENT COLLABORATION IN MEDICAL REASONING :

Multi-agent systems divide a reasoning task among agents with defined roles. In MedAgents, role-specific LLM agents analyze a medical question independently, discuss their findings, summarize the discussion and reach a decision in a zero-shot setting [2], so that each agent can concentrate on a distinct perspective before synthesis. KG4Diagnosis pairs a general-practitioner-style agent with specialized agents and an automatically constructed medical knowledge graph [4], which represents relationships between symptoms and diseases explicitly at the cost of additional knowledge-engineering work.

The adaptive CDSS of Öğdü et al. couples specialist agents with biomedical RAG, web evidence, data fusion and a confidence-triggered feedback loop [1], and it is closely related to MedIntel in its emphasis on modularity and confidence-aware processing. MACD adds self-learned clinical knowledge to multi-agent diagnosis [10]. Collaboration has a cost, because multi-agent systems can require more inference calls, retrieval steps and serialization overhead than a single predictive model. MedIntel keeps the benefit of specialization but runs independent agents in parallel, uses deterministic preprocessing

where possible, reserves generative synthesis for the final stages, and replaces open-ended conversation with fixed output contracts.

2.5 REVIEW OF TECHNICAL STACKS FOR CLINICAL AI SYSTEMS :

Technical stacks for a clinical AI prototype are assessed here against three criteria: the output of each component must be inspectable, patient data must be able to stay on site, and every run must be reproducible. Python 3.11 suits these criteria because of its mature libraries. Tabular models from scikit-learn and XGBoost expose class probabilities and feature importance, sentence-transformers embeddings in a local FAISS index keep retrieval inspectable without a separate database server, and rules stored as JSON remain readable and versionable for the Knowledge Curator.

For the language model, a configurable endpoint can call either a hosted API or a local instruction-tuned model served with Ollama, so case content can be kept inside the institution when required. No agent-orchestration framework is needed, because the agents are plain Python classes that exchange typed objects, and the standard concurrent.futures and asyncio modules are enough to process independent cases in parallel. FastAPI and Streamlit keep the REST backend and the clinician dashboard in the same language, while pinned library versions and recorded random seeds make each run repeatable.

2.6 RESEARCH GAP IDENTIFICATION :

Read together, Table 2.2 and the preceding sections show where the gap lies. The multi-agent frameworks [1], [2], [4], [10] build their reasoning around LLM agents, the retrieval-grounded systems [3], [8], [9] concentrate on grounding the generator in external evidence, and the reviews [6], [7], [11], [12] call for stronger clinical validation. Together the studies suggest that several evidence sources help, yet none offers a simple, auditable architecture in which tabular machine learning, deterministic rules, retrieval and generative synthesis meet at a common patient-level interface. Evaluation forms a second gap: a multi-component system needs evidence that each component adds value and that its confidence is calibrated, which a single benchmark accuracy cannot show. MedIntel is designed to address both gaps, and Chapters 3 and 4 turn them into requirements and an architecture.

SUMMARY :

Chapter 2 reviewed twelve works on AI-based clinical decision support and compared them in Table 2.2. The review found that single-model designs are hard to audit and that adding

agents improves specialization while making the origin of a recommendation harder to trace, which leads to the needs for evidence separation and human oversight. Learned models, explicit rules and retrieval were discussed as complementary techniques, and the planned Python stack was justified. Finally, the chapter identified the two gaps MedIntel targets: no common, auditable patient-level interface for these evidence sources, and no evaluation of what each component adds.

CHAPTER 03

SOFTWARE & HARDWARE REQUIREMENT SPECIFICATION (SRS)

This chapter specifies what MedIntel must do and under what conditions. Written at the end of Project Stage I, before any code exists, it is the agreed reference for the implementation and testing planned in Stage II. It fixes the scope of the software, its user classes and the assumptions behind the design, and then states the functional requirements FR1 to FR9, the external interfaces, the platform, and the non-functional requirements NFR1 to NFR7 that later chapters cite by number.

3.1 INTRODUCTION TO SYSTEM REQUIREMENTS :

3.1.1 PROJECT SCOPE :

In requirement terms, MedIntel is a single application with a clear input and output boundary. On the input side it accepts de-identified patient records as CSV files, tabular datasets or event logs, together with the rulebase, knowledge collection and configuration that the Knowledge Curator and System Administrator maintain. On the output side it returns a structured DecisionReport and an AuditRecord for every case, and it accepts the Clinician’s review in return. Everything between these boundaries, from record validation to the curator-approved knowledge update, belongs to the software to be built; the exclusions of Section 1.5 apply unchanged.

3.1.2 USER CLASSES AND CHARACTERISTICS :

Four classes of users will work with MedIntel. They differ in technical background, but more importantly in the authority each holds over patient data, rules and knowledge. The Clinician is the primary user and the only one who decides what happens to a recommendation; the other three keep the input data, the rulebase and the platform in a state the Clinician can rely on. Table 3.1.2 summarizes the goal and key characteristics of each class.

Table 3.1.2 : User Classes And Characteristics

User Class	Primary Goal	Key Characteristics
Clinician	Obtain a traceable recommendation for a patient case and accept, reject or annotate it.	Primary user with clinical expertise but little programming knowledge; works only through the dashboard; keeps final decision authority.
Data Administrator	Import and de-identify patient records and correct validation errors so that clean cases enter the pipeline.	Comfortable with CSV files, tabular data and the input schema; has no role in clinical decisions.
Knowledge Curator	Maintain the rulebase and approve which validated outcomes and references enter the knowledge store.	Has clinical or domain knowledge and can edit JSON rule files; acts as the gatekeeper for every knowledge update.
System Administrator	Configure models, fusion weights and thresholds, and monitor the audit log and system health.	Technical user with Python and deployment skills; manages model and index versions and user access.

3.1.3 ASSUMPTIONS AND TECHNICAL DEPENDENCIES :

A1: Patient records will be de-identified before they reach MedIntel, and the data used in Stage II will be split into training, validation and held-out test partitions with no patient appearing in more than one partition.

A2: Development and deployment machines will run Python 3.11 with pandas, pydantic, scikit-learn, XGBoost, sentence-transformers, FAISS, FastAPI and Streamlit, installed from a pinned requirements file so that every team member works with the same versions.

A3: An instruction-tuned LLM will be reachable through a configurable endpoint, a hosted API or a local model served with Ollama. If it cannot be reached, the pipeline still returns the ML, rule and retrieval evidence and records the synthesis step as failed.

A4: The knowledge store will be seeded with guideline excerpts, articles and institution-approved references whose sources are known, and the Knowledge Curator will supply the first version of the rulebase as a JSON file.

A5: MedIntel will be used only as an advisory tool. A qualified clinician reviews every report, and no recommendation is treated as a diagnosis or acted upon without that review.

3.2 FUNCTIONAL REQUIREMENTS (FRS) :

The functional requirements follow a patient case through the system, from the upload of a record to the Clinician's decision on the finished report. Each requirement maps to a specific part of the design in Chapter 4, which keeps the link between what the system must do and where it is done easy to check when the Stage II test cases are run. The system must provide the capabilities listed in Table 3.2.

Table 3.2 : Functional Requirements (FRs)

FR No.	Requirement	Description
FR1	Patient Data Ingestion And Validation	The system must accept patient data as CSV files, tabular datasets or event logs, check each record for data types, missing fields, duplicates, unit consistency and identifier integrity, and report every rejected record with its reason.
FR2	Case Construction And Parallel Scheduling	The system must build one PatientCase per patient (structured features, text context, event logs and source metadata), queue the valid cases, and process several of them concurrently, each in an isolated context.
FR3	Machine-Learning Analysis	The system must apply a configurable trained model (logistic regression, random forest or gradient boosting) to the structured features and return the prediction, class probabilities, feature importance and model version.
FR4	Rule-Based Validation	The system must evaluate each case against a versioned rulebase of thresholds, co-occurring findings and required-field checks, recording every triggered rule with its action (flag, elevate or suppress) and reason.
FR5	Knowledge Retrieval	The system must derive a query from the case, search the indexed knowledge collection, and return the top-*k* passages with source identifiers and similarity scores, or state that no sufficiently relevant evidence was found.
FR6	Clinical Context Synthesis	The system must send the ML, rule and retrieval outputs to the LLM Service, obtain a short clinical context summary that names any conflict between components, and list any statement the supplied evidence does not support.
FR7	Decision Fusion And Confidence Scoring	The system must combine the component outputs into a candidate decision with an explicit confidence score that rewards agreement and penalizes disagreement, keeping each component score and the disagreement value for inspection.

FR No.	Requirement	Description
FR8	Explanation Report And Audit Trail	The system must generate a structured report (salient findings, model prediction, triggered rules, evidence references, conflicts, confidence and recommendation) and store an audit record with model and rule versions, retrieval identifiers, timestamp and final status.
FR9	Clinician Review And Controlled Knowledge Update	The system must let the Clinician accept, reject or annotate each report, record that decision in the audit trail, and re-index into the knowledge store only validated outcomes approved by the Knowledge Curator.

3.3 EXTERNAL INTERFACE REQUIREMENTS :

3.3.1 USER INTERFACES :

The main user interface is a Streamlit dashboard for the Clinician. A case list shows each case with its status and confidence, and opening a case displays its DecisionReport in a fixed order, from patient identifier, salient findings and model prediction through triggered rules, evidence references and conflicts to confidence and recommendation. Cases with low confidence or reported conflicts are highlighted, and controls under the report let the Clinician accept, reject or annotate it. Role-specific pages let the Data Administrator upload files and review validation errors, the Knowledge Curator approve pending updates, and the System Administrator change settings and browse the audit log.

3.3.2 SOFTWARE INTERFACES :

Each external library or service is reached through a single MedIntel class rather than from many places in the code, so replacing it affects only one module (NFR3). This matters most for the model and the LLM, which the design leaves configurable on purpose, because the best choice depends on the dataset and clinical task selected in Stage II. The planned software interfaces are:

- **pandas and pydantic :** Read the tabular input and enforce the PatientCase schema in the SchemaValidator.
- **scikit-learn and XGBoost :** Provide the logistic regression, random forest and gradient boosting models of the MLAnalysisAgent.
- **sentence-transformers and FAISS :** Embed queries and reference passages and search the KnowledgeStore index.

- **LLM Service :** An instruction-tuned model behind a configurable endpoint (a hosted API or a local Ollama model), used for synthesis and explanation text.
- **FastAPI and Streamlit :** Serve the REST backend and the clinician dashboard.
- **SQLite and PostgreSQL :** Store audit records during development and deployment respectively.

3.3.3 COMMUNICATION INTERFACES :

The dashboard and backend will communicate through a FastAPI REST interface exchanging JSON over HTTP. Four endpoints cover the workflow: POST /cases/upload submits a file, POST /cases/{case_id}/analyze starts the pipeline for a case, GET /cases/{case_id}/report returns its DecisionReport, and POST /cases/{case_id}/review records the Clinician's decision. Inside the backend, agents pass typed Python objects, not network messages. The only outbound call goes to the LLM Service, over HTTPS for a hosted endpoint or to a local port under Ollama, and it carries only the de-identified fields that synthesis needs.

3.4 HARDWARE AND SOFTWARE REQUIREMENTS FOR DEVELOPMENT AND DEPLOYMENT :

MedIntel is planned to be developed on the team's laptops and demonstrated on a single workstation. Its heaviest parts are the embedding model, the FAISS index and, when a local model replaces the hosted endpoint, the LLM. The minimum configuration therefore assumes a hosted endpoint; the recommended one can run the whole pipeline offline and is the hardware for the NFR4 latency target. Git, GitHub, VS Code and Jupyter are used throughout. Table 3.4 lists both configurations.

Table 3.4 : Hardware And Software Requirements For Development And Deployment

Requirement	Minimum	Recommended
Operating System	64-bit Windows 10, Ubuntu 22.04 or macOS 13	64-bit Ubuntu 22.04 LTS or Windows 11
Processor	Quad-core x86-64 or ARM64, 2.5 GHz	Eight-core, 3.0 GHz or faster
RAM	8 GB	16 GB (32 GB if a local LLM runs on the same machine)
Storage	20 GB free SSD space	50 GB free SSD space for model weights, the vector index and audit data

Requirement	Minimum	Recommended
GPU (Optional)	Not needed when a hosted LLM endpoint is used	NVIDIA GPU with at least 8 GB VRAM for a local LLM and faster embedding
Python Version	Python 3.11	Python 3.11 in an isolated virtual environment
Key Libraries	pandas, pydantic, scikit-learn, XGBoost, sentence-transformers, FAISS, FastAPI, Streamlit, pytest	The same set, with PostgreSQL in place of SQLite for audit records and Ollama for a local LLM

3.5 NON-FUNCTIONAL REQUIREMENTS (NFRS) :

The non-functional requirements describe the qualities MedIntel must show while performing its functions. In clinical decision support they matter as much as the functions: a correct recommendation that cannot be traced to its evidence, or one built partly from another patient's data, would still be unacceptable. Measurable qualities carry a target, which is a design goal to be checked in Stage II rather than a measured value. Table 3.5 lists the seven NFRs.

Table 3.5 : Non-Functional Requirements (NFRs)

NFR No.	Requirement	Description / Target
NFR1	Traceability And Auditability	Every recommendation must be traceable to the input features, model version, triggered rules and retrieved evidence behind it. Target: every report has a complete AuditRecord.
NFR2	Privacy And Patient Isolation	Only de-identified records are processed, and no evidence, passage or rule result from one case is reused in another unless it belongs to the shared knowledge base.
NFR3	Modularity And Maintainability	Each agent works behind a fixed input and output contract, so a model, rulebase, index or LLM can be replaced without changing the other modules.
NFR4	Performance	Target: one case processed end to end within 30 seconds on the recommended hardware, with the ML and rule stages each under 1 second; independent agents run in parallel.
NFR5	Reliability	Malformed or missing data must never crash the pipeline. Every failure is logged by category: data-validation error, model error, retrieval failure, rule conflict, synthesis error or interface problem.
NFR6	Reproducibility	Dataset, preprocessing configuration, model, rulebase, index and prompt versions and random seeds are recorded so that any run can be repeated from a clean environment.

NFR No.	Requirement	Description / Target
NFR7	Usability	Target: a clinician can read and understand a report in under two minutes, with conflicts and low confidence clearly marked.

SUMMARY :

Chapter 3 specified the requirements that MedIntel must satisfy. The software boundary was fixed between de-identified patient records and audited, reviewable reports, and four user classes were defined, with the Clinician as the only one who decides on a recommendation. Nine functional requirements, FR1 to FR9, follow a case from ingestion to clinician review and controlled knowledge update, supported by the stated interfaces and a Python 3.11 platform. Seven non-functional requirements, led by traceability and patient isolation, set the targets that the design in Chapter 4 must meet and Stage II testing will check.

CHAPTER 04

SYSTEM DESIGN AND ARCHITECTURE

This chapter turns the requirements of Chapter 3 into a design. It covers the architecture and data flow of a patient case, the agent output contracts, the fusion logic behind each confidence score, and parallel execution with its controlled knowledge update. UML diagrams then model the structure and behavior of the system, and the chapter closes with the data structures of the rulebase and the decision report.

4.1 SYSTEM ARCHITECTURE AND COMPONENT OVERVIEW :

4.1.1 HIGH-LEVEL ARCHITECTURE DIAGRAM :

The architecture assigns each of the project's four design constraints to a specific component. The CaseScheduler provides patient isolation by giving every case its own context, the RuleValidationAgent provides deterministic guardrails by applying explicit rules before any generative step, the KnowledgeRetrievalAgent provides evidence grounding by returning each passage with its source identifier, and the ClinicianDashboard provides human oversight by placing every recommendation before a clinician. Fig 4.1.1 shows the resulting high-level architecture.

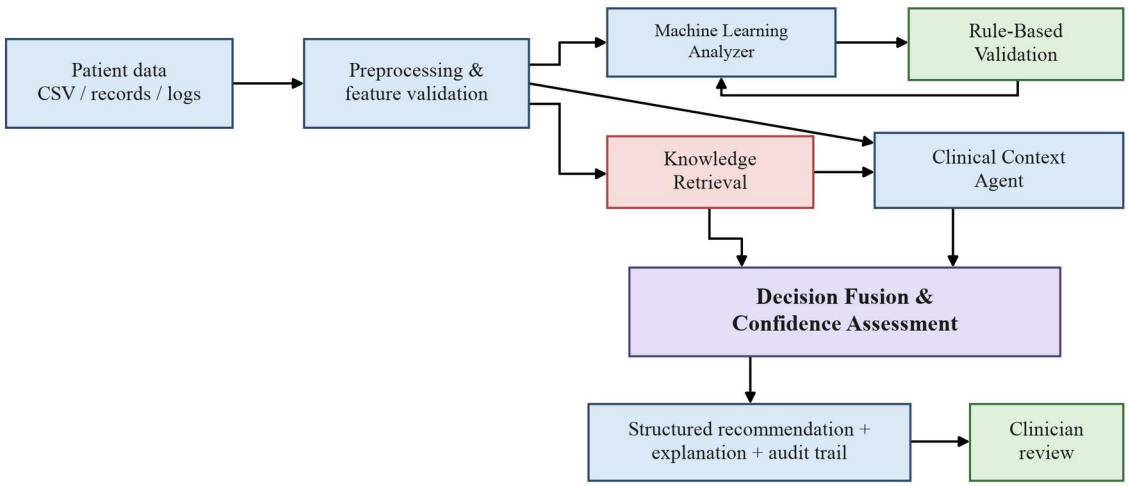


Fig 4.1.1 : MedIntel High-Level System Architecture

Patient data from CSV files, records or logs passes through preprocessing and feature validation, which sends the case to the Machine Learning Analyzer, to Knowledge Retrieval and, as context, to the Clinical Context Agent. The paired arrows to and from Rule-Based Validation show each learned prediction being checked against the rules. Retrieved

evidence reaches the Clinical Context Agent and, with the synthesized context, Decision Fusion & Confidence Assessment, whose output becomes the structured recommendation, explanation and audit trail passed to clinician review. Only the main paths are drawn; Table 4.2.1 lists every agent input. The key components are:

- **DataIngestionService and SchemaValidator :** Implement preprocessing and feature validation, building one PatientCase per patient (FR1, FR2).
- **CaseScheduler and Orchestrator :** Queue isolated cases and coordinate the agents (FR2); as control classes they are not drawn here, though the scheduler appears in Fig 4.2.3.
- **MLAnalysisAgent and RuleValidationAgent :** Implement the analyzer and rule blocks and return an MLResult and a RuleResult (FR3, FR4).
- **KnowledgeRetrievalAgent :** Searches the KnowledgeStore and returns EvidenceItem objects (FR5).
- **ClinicalContextAgent and DecisionFusionEngine :** Synthesize the evidence through the LLM Service and fuse it into a FusionResult (FR6, FR7).
- **ExplanationGenerator, AuditLogger and ClinicianDashboard :** Produce the DecisionReport and AuditRecord and present the report for review (FR8, FR9).

4.1.2 DATA FLOW DIAGRAM (DFD) OF PATIENT CASE PROCESSING :

The data flow diagrams trace how patient data moves through MedIntel and where it is stored. At Level 0 the system is one process with five external entities: the Data Administrator supplies records and receives validation errors, the Clinician selects cases, receives reports and returns review decisions, the Knowledge Curator supplies rules and approvals, the System Administrator sets the configuration and reads the audit log, and the LLM Service returns generated text for each synthesis request. Fig 4.1.2 (a) shows this context diagram.

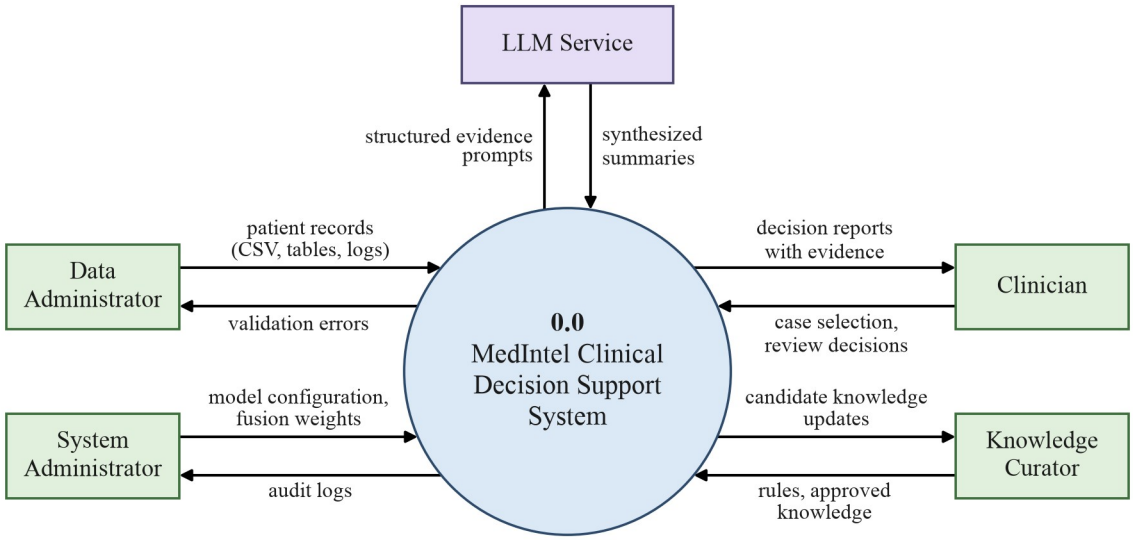


Fig 4.1.2 (a) : Level 0 Data Flow Diagram (Context Diagram)

Fig 4.1.2 (b) decomposes this process into eight numbered processes that share five data stores, D1 to D5. During analysis the model registry, rulebase and knowledge store are only read and the audit log is only appended to, so an analysis can never alter the evidence it used. The numbered steps after the figure trace one case through the diagram.

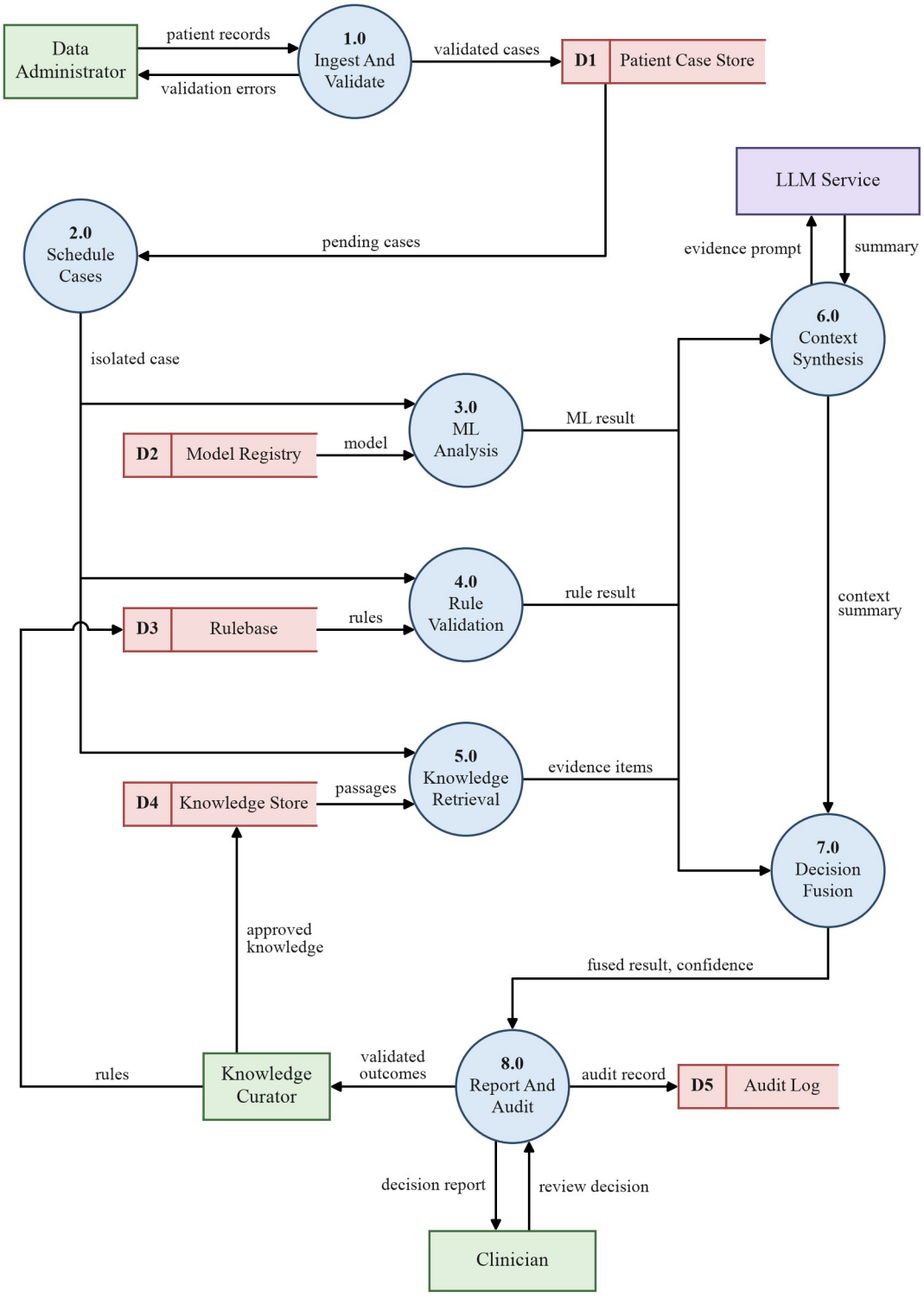


Fig 4.1.2 (b) : Level 1 Data Flow Diagram

1. **Ingestion:** 1.0 Ingest And Validate writes valid cases to D1 Patient Case Store and returns an error for each rejected record.
2. **Analysis:** 2.0 Schedule Cases takes each case from D1 and passes it, in isolation, to 3.0 ML Analysis, 4.0 Rule Validation and 5.0 Knowledge Retrieval, which read D2 Model Registry, D3 Rulebase and D4 Knowledge Store respectively.

3. **Synthesis and fusion:** 6.0 Context Synthesis calls the LLM Service, and 7.0 Decision Fusion produces the decision and its confidence.
4. **Reporting and review:** 8.0 Report And Audit sends the DecisionReport to the Clinician, appends audit records, including the review decision, to D5 Audit Log, and passes validated outcomes to the Knowledge Curator, who alone writes approved knowledge to D4 and rules to D3.

For readability, Level 1 leaves out the administrative flows shown at Level 0: the model configuration and fusion weights set by the System Administrator, the audit log returned to that administrator, and the Clinician's case selection. These flows use the same processes and stores as the analysis path, so leaving them out does not change the structure of the diagram.

4.2 AGENT DESIGN DETAILS :

4.2.1 AGENT RESPONSIBILITIES AND OUTPUT CONTRACTS :

The analysis agents read the same PatientCase, and every component returns a typed result object instead of free text. These output contracts make the pipeline auditable, because each report field traces back to the component that produced it, and modular, because a component can be swapped while its contract stays fixed (NFR3). Table 4.2.1 lists the input, output and key responsibility of each agent.

Table 4.2.1 : Agent Responsibilities And Output Contracts

Agent	Input	Output	Key Responsibility
MLAnalysisAgent	PatientCase (structured features)	MLResult: prediction, class probabilities, feature importance, model version	Detect learned patterns; the output is evidence, not a diagnosis.
RuleValidationAgent	PatientCase and RuleBase	RuleResult: triggered rule ids, flags, action, reason, rulebase version	Apply deterministic thresholds and policies; never override a prediction without a recorded reason.
KnowledgeRetrievalAgent	Query built from the PatientCase	List of EvidenceItem: source_id, passage, similarity score	Ground the case in approved references, or report that nothing relevant was found.

Agent	Input	Output	Key Responsibility
ClinicalContextAgent	PatientCase and the three analysis results	ContextSummary: summary text, conflicts, unsupported statements	Reconcile the evidence through the LLM Service, state conflicts and add no unsupported observation.
DecisionFusionEngine	Analysis results and context summary	FusionResult: decision *y*, confidence *C*, disagreement *D*, component scores	Weigh the evidence with equation (4.2) and keep every component score visible.
ExplanationGenerator	PatientCase and FusionResult	DecisionReport: findings, prediction, rules, evidence, conflicts, confidence, recommendation, status	Turn technical outputs into a structured report for clinician review.
AuditLogger	Report and its review status	AuditRecord: model version, timestamp, rule version, retrieval ids, final status	Keep an append-only trace of each report and review decision.

4.2.2 DESIGN OF THE DECISION FUSION AND CONFIDENCE LOGIC :

The fusion logic starts from a formal statement of the problem. A patient case is a heterogeneous input X of structured attributes such as age, laboratory values and vital signs, text-based clinical context, and event or log observations. Given a deterministic rulebase R and an indexed knowledge source K, MedIntel computes

$$(y, E, z) = F(X, R, K) \tag{4.1}$$

where y is the candidate decision, E is the evidence set behind it and z is the explanation shown to the clinician. Since y depends on R and K as well as on X, the versions of both are stored with every result (NFR6). How far y can be trusted is expressed by an explicit confidence score rather than by the language model's own judgment:

$$C = \alpha M + \beta R + \gamma K - \lambda D \tag{4.2}$$

where M is the normalized support of the ML model, R is rule consistency, K is retrieval support and D is the disagreement among components; here R and K are scores derived from the rulebase and the knowledge source. For the binary risk task planned for the first prototype, M is the probability of the predicted class, R is 1 unless an elevating or suppressing rule contradicts that prediction (then 0), K is the mean similarity of the retrieved

passages (0 when none is relevant), and *D* is the number of conflicts reported by the ClinicalContextAgent divided by three, the number of evidence sources, capped at 1.

The equation makes the fusion explicit and testable without claiming clinical validity. Its weights α, β, γ and λ, like the decision threshold, will be tuned only on a separate validation set. A triggered elevate or suppress rule takes precedence over the learned prediction, always with its reason recorded, and when *C* falls below the threshold the decision is reported as uncertain and the case is marked low confidence. Setting a weight to zero removes that component from the fusion, which is how most of the planned ablation runs can be configured.

4.2.3 MULTI-PATIENT PARALLEL EXECUTION AND KNOWLEDGE UPDATE LOOP :

MedIntel is designed to analyze a batch of patients at once. The CaseScheduler queues validated records and gives each case its own context, so nothing from one patient's analysis can enter another's (NFR2). Within a case, the ML, rule and retrieval agents can run concurrently, since rule evaluation reads only the case data, while synthesis and fusion wait for all of that case's evidence. Fig 4.2.3 shows four cases processed in this way.

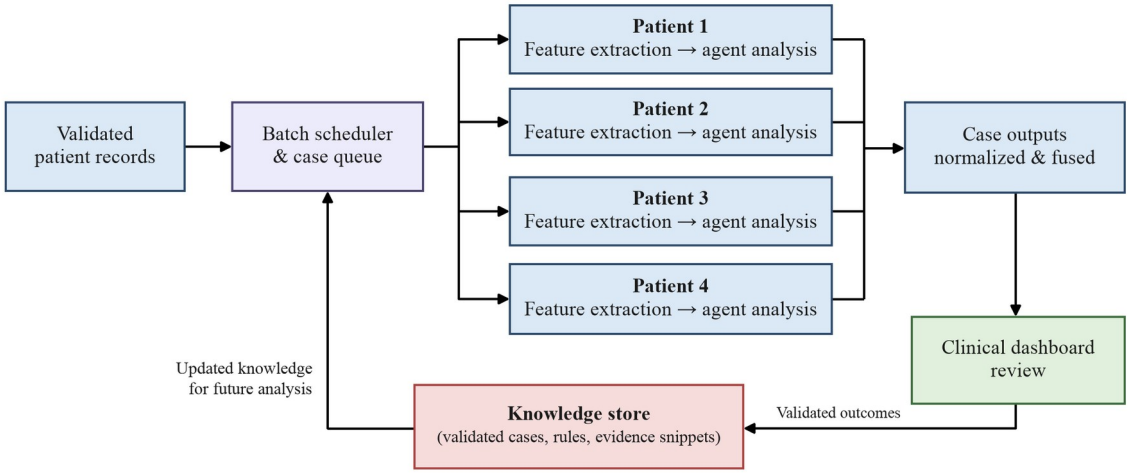


Fig 4.2.3 : Multi-Patient Parallel Processing And Knowledge Update Loop

In the figure, the batch scheduler and case queue release each patient into its own lane of feature extraction and agent analysis, and the lanes meet only where case outputs are normalized and fused for clinical dashboard review. Validated outcomes then flow into the knowledge store of validated cases, rules and evidence snippets, which returns updated knowledge to the scheduler for future analysis. This loop is narrow on purpose: KnowledgeStore.add_validated is the only write path into the store, rule changes go to the

RuleBase, and the Knowledge Curator approves both. Each approved change also creates a new index or rulebase version, so an earlier report can still be reproduced against the knowledge it actually used (NFR6).

4.3 UNIFIED MODELING LANGUAGE (UML) DIAGRAMS :

4.3.1 USE CASE DIAGRAM FOR PRIMARY ACTORS :

The use case diagram shows MedIntel from the side of its actors. The Clinician, the primary actor, uploads patient records, runs a case analysis, views the decision report, inspects the evidence and audit trail, and reviews and annotates the recommendation (FR8, FR9). Run Case Analysis includes ML Analysis, Rule Validation, Knowledge Retrieval, Context Synthesis and Decision Fusion (FR3 to FR7), with the LLM Service attached to Context Synthesis as an external actor. Upload Patient Records, shared with the Data Administrator, includes Validate And Normalize Data, which the Data Administrator can also run directly (FR1, FR2). The Knowledge Curator manages the rulebase and approves knowledge updates (FR4, FR9), and the System Administrator configures models and fusion weights and monitors audit logs. Fig 4.3.1 shows the diagram.

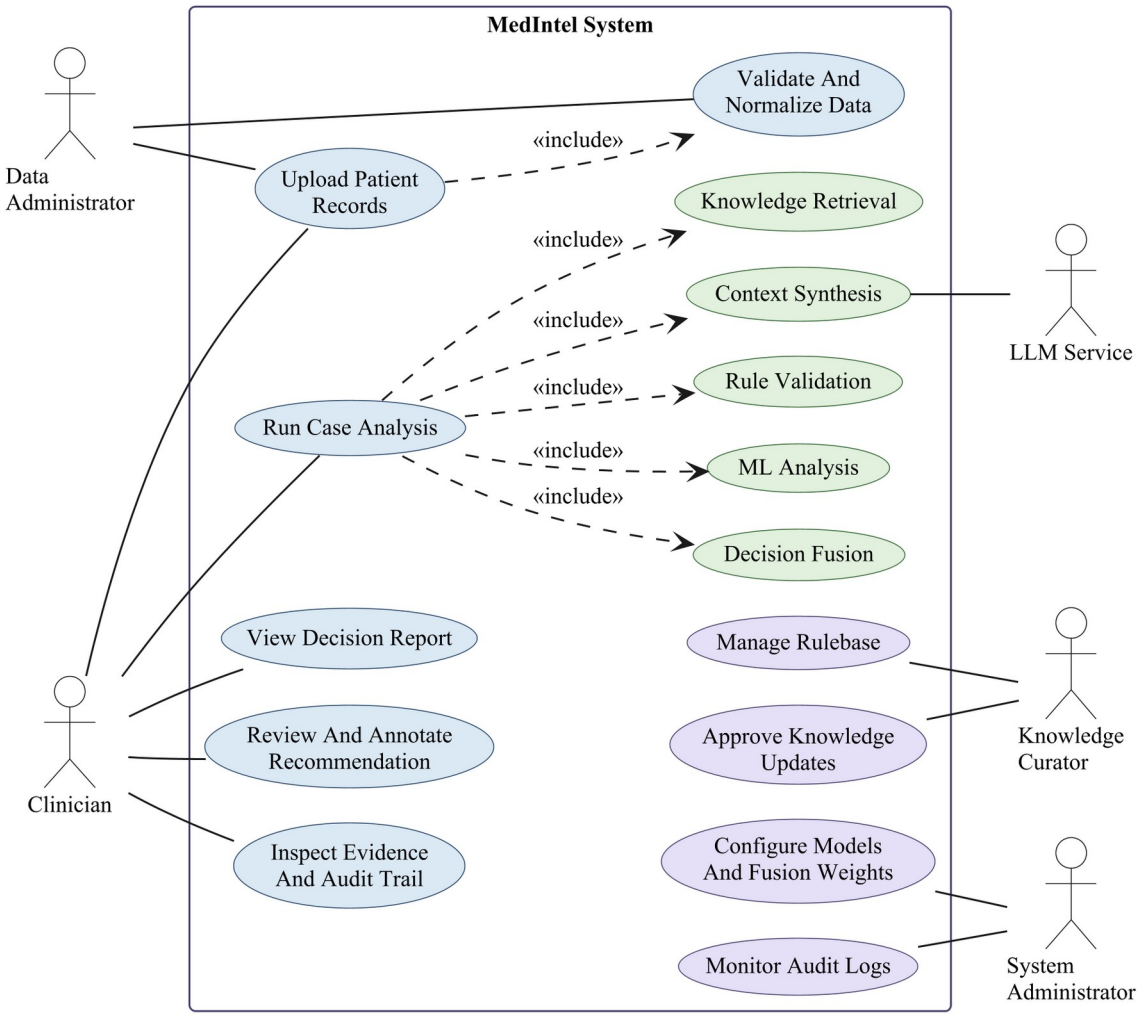


Fig 4.3.1 : Use Case Diagram For Primary Actors

4.3.2 CLASS DIAGRAM OF CORE SYSTEM OBJECTS :

The class diagram models the static structure of the core objects. Two control classes coordinate the work: the CaseScheduler, with submit(cases) and run(max_workers), and the Orchestrator, whose analyze(case) runs the agents for one case and returns a DecisionReport. Each agent exposes one main operation, such as evaluate(case) on the RuleValidationAgent, which holds a RuleBase of Rule objects. The data classes, PatientCase, MLResult, RuleResult, EvidenceItem, ContextSummary, FusionResult, DecisionReport and AuditRecord, carry the contracts of Table 4.2.1. In Fig 4.3.2, purple marks the two control classes, blue the services and agents, red the rulebase and knowledge store, and green the data objects.

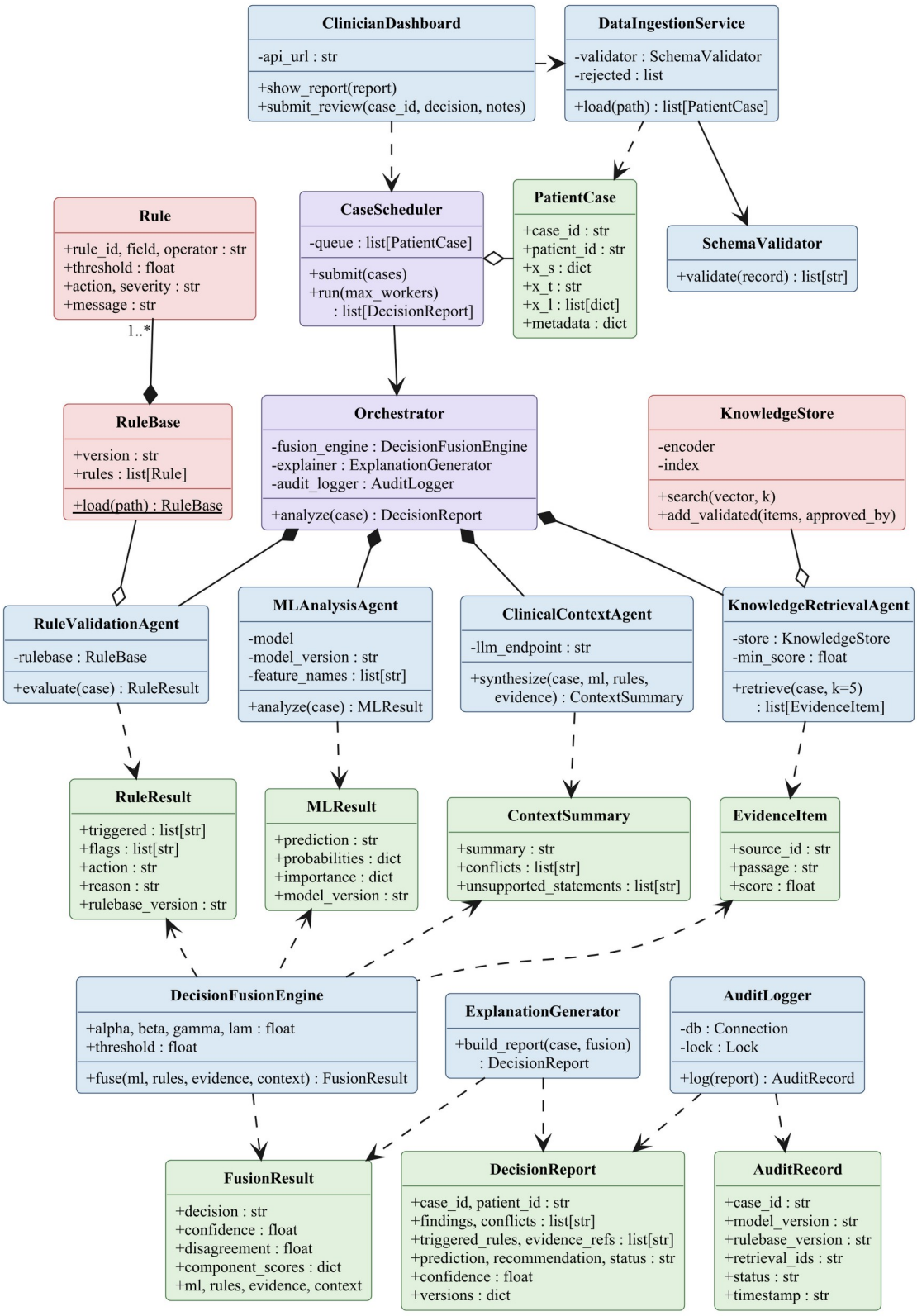


Fig 4.3.2 : Class Diagram Of Core System Objects

4.3.3 SEQUENCE DIAGRAM FOR PATIENT CASE ANALYSIS :

The sequence diagram follows one case from the Clinician's request to the recorded review. Its messages use the method names of the class diagram, so the two views can be checked against each other. Because thirteen lifelines cannot be read in a single figure, the diagram is

split using a standard UML interaction reference. Fig 4.3.3 (a) shows the top-level interaction from upload to review, and its “ref Analyze One Case” frame is expanded in Fig 4.3.3 (b), which is the body of Orchestrator.analyze(case). The three analysis calls are independent and run in parallel, while synthesis waits for all of them. The key interaction sequence is listed after the figures.

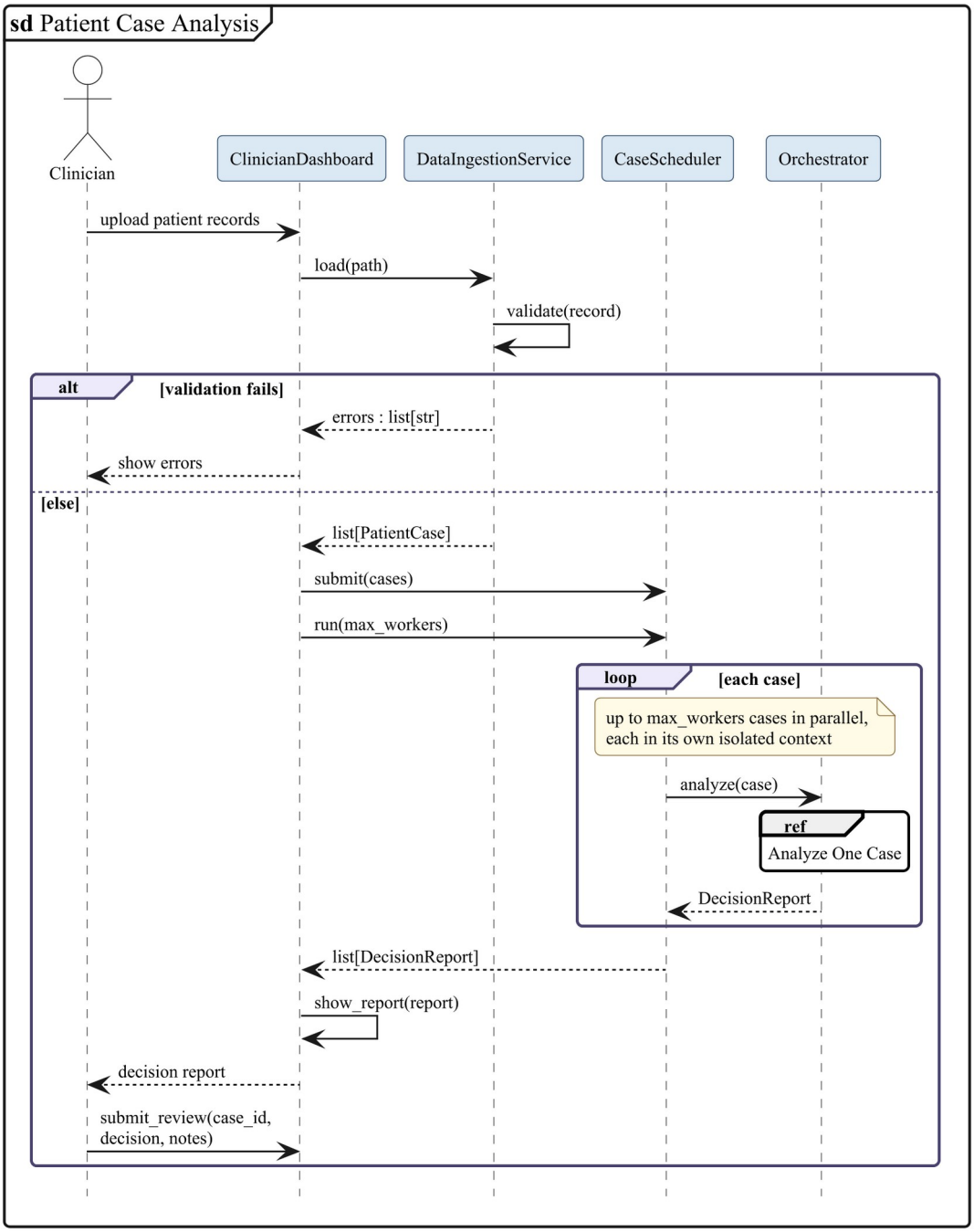


Fig 4.3.3 (a) : Sequence Diagram For Patient Case Analysis

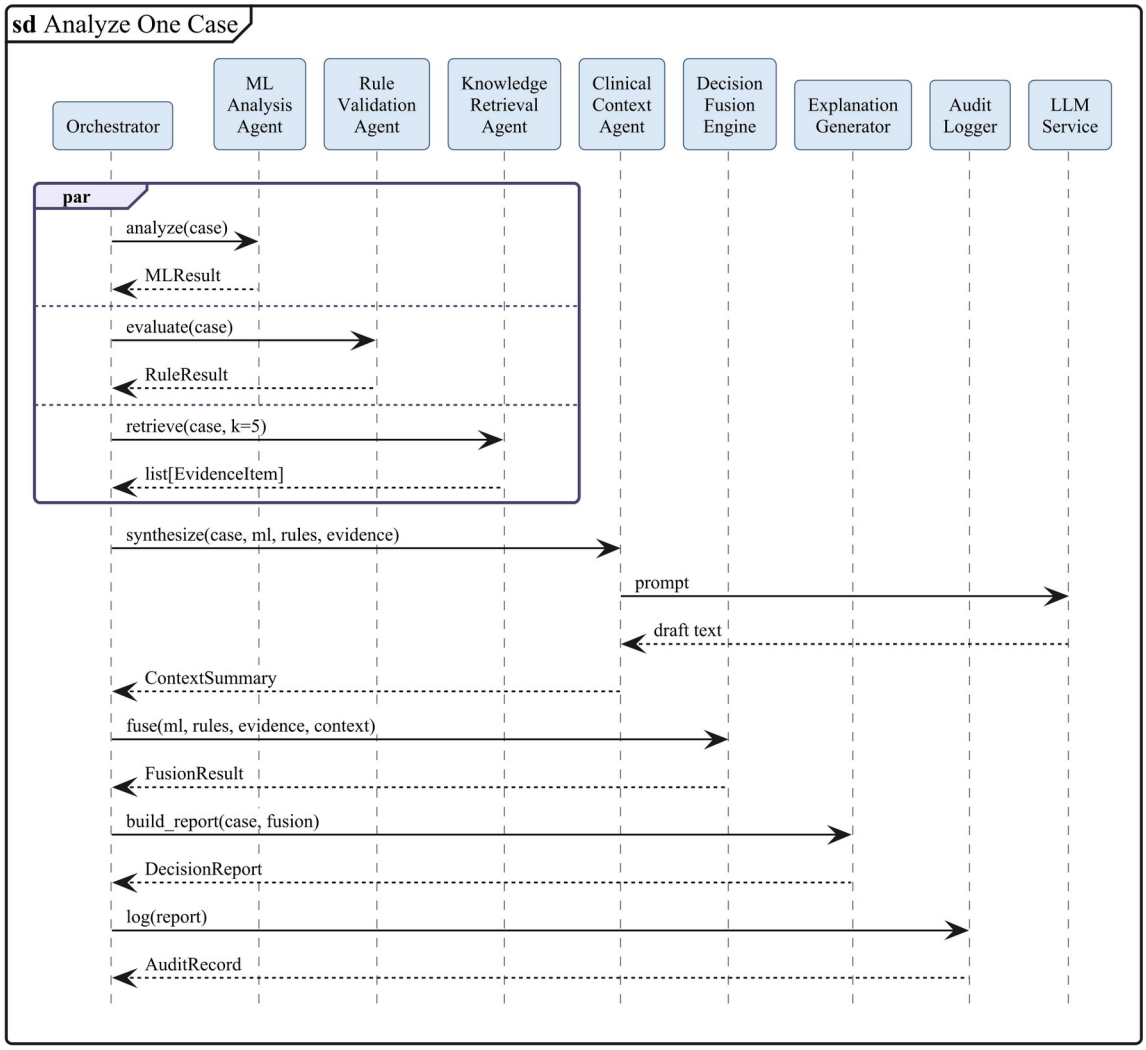


Fig 4.3.3 (b) : Sequence Diagram For Analyzing One Case

1. The Clinician uploads patient records on the ClinicianDashboard, which calls DataIngestionService.load(path); each record is checked with validate(record).
2. If validation fails, the list of errors is returned and shown to the Clinician. Otherwise the list of PatientCase objects is passed to the CaseScheduler through submit(cases) and run(max_workers).
3. For each case, with up to max_workers cases in parallel and each in its own isolated context, the CaseScheduler calls Orchestrator.analyze(case), which runs the interaction of Fig 4.3.3 (b).
4. Inside it, MLAnalysisAgent.analyze(case), RuleValidationAgent.evaluate(case) and KnowledgeRetrievalAgent.retrieve(case, k=5) run in parallel and return an MLResult, a RuleResult and a list of EvidenceItem objects.
5. ClinicalContextAgent.synthesize(case, ml, rules, evidence) sends an evidence prompt to the LLM Service and returns a ContextSummary, after which DecisionFusionEngine.fuse(ml, rules, evidence, context) returns the FusionResult.

6. ExplanationGenerator.build_report(case, fusion) creates the DecisionReport, and AuditLogger.log(report) stores its AuditRecord before the report is returned.
7. The CaseScheduler returns the list of DecisionReport objects, ClinicianDashboard.show_report(report) displays each one, and submit_review(case_id, decision, notes) records the Clinician's decision.

4.4 DATA STRUCTURE DESIGN :

4.4.1 RULEBASE DATA STRUCTURE :

Rules are kept outside the code in a versioned file, rulebase.json, so the Knowledge Curator can edit them without touching Python and every RuleResult can name its rulebase version (NFR1, NFR6). Each rule tests one PatientCase field with an operator and, where needed, a threshold, and carries an action, a severity and a message copied into the report when it fires; co-occurring findings use derived fields prepared at ingestion. The excerpt from rulebase.json below shows one rule per action type, with illustrative thresholds.

JSON

```
{
 "version": "1.0.0",
 "approved_by": "knowledge_curator",
 "rules": [
  {
   "rule_id": "R-001",
   "field": "systolic_bp",
   "operator": ">=",
   "threshold": 180,
   "action": "elevate",
   "severity": "high",
   "message": "Systolic blood pressure at or above 180 mmHg; urgent review advised."
  },
  {
   "rule_id": "R-002",
   "field": "creatinine",
   "operator": "missing",
   "threshold": null,
   "action": "flag",
   "severity": "medium",
   "message": "Serum creatinine missing; kidney-related rules cannot be checked."
  },
  {
   "rule_id": "R-003",
   "field": "age",
   "operator": "<",
   "threshold": 18,
   "action": "suppress",
   "severity": "medium",
   "message": "Model intended for adults; recommendation withheld under age 18."
  }
```

```
 ]
}
```

4.4.2 JSON SCHEMA FOR THE DECISION REPORT :

The DecisionReport is the object a clinician reads and the response of GET /cases/{case_id}/report. Its structure is fixed by a JSON Schema, so the ExplanationGenerator, the dashboard and the Stage II tests share one set of fields, and a report that lacks a required field is rejected before display. The report also carries the model, rulebase and index versions, which the AuditLogger copies into the AuditRecord with a timestamp. The schema, stored as decision_report.schema.json, is shown below.

JSON

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "title": "DecisionReport",
 "type": "object",
 "required": [
  "case_id", "patient_id", "findings", "prediction", "triggered_rules",
  "evidence_refs", "conflicts", "confidence", "recommendation", "status", "versions"
 ],
 "properties": {
  "case_id": {"type": "string"},
  "patient_id": {"type": "string"},
  "findings": {"type": "array", "items": {"type": "string"}},
  "prediction": {"type": "string"},
  "triggered_rules": {"type": "array", "items": {"type": "string"}},
  "evidence_refs": {"type": "array", "items": {"type": "string"}},
  "conflicts": {"type": "array", "items": {"type": "string"}},
  "confidence": {"type": "number"},
  "recommendation": {"type": "string"},
  "status": {"enum": ["pending_review", "accepted", "rejected", "annotated", "failed"]},
  "notes": {"type": "string"},
  "versions": {
   "type": "object",
   "properties": {
    "model": {"type": "string"},
    "rulebase": {"type": "string"},
    "index": {"type": "string"}
   }
  }
 }
}
```

SUMMARY :

Chapter 4 set out the design of MedIntel. The architecture keeps learned inference, rule checks and retrieval apart until the synthesis and fusion stages, where the confidence function of equation (4.2) weighs their support against disagreement. The data flow diagrams, agent contracts and parallel execution scheme showed how each case moves in

isolation and how knowledge changes only with clinician validation and curator approval. The UML diagrams and the JSON structures of the rulebase and decision report fixed the objects and data that Chapter 5 implements.

CHAPTER 05

IMPLEMENTATION DETAILS

This chapter explains how the modules designed in Chapter 4 are to be implemented in Project Stage II. After the planned tools, it gives reference Python code for each module in pipeline order, named as in the class diagram of Section 4.3.2. None of the listings has been run on patient data; they fix the interfaces and logic that Stage II will build and test.

5.1 INTRODUCTION TO IMPLEMENTATION TOOLS AND ENVIRONMENT :

MedIntel is to be written entirely in Python 3.11. Each library has a narrow job in the code: pandas and pydantic load and check input files, scikit-learn and XGBoost provide the classifiers, sentence-transformers and FAISS handle evidence search, and a small rule engine written for the project evaluates the JSON rulebase. FastAPI serves the backend and Streamlit the dashboard. Coding is planned in VS Code and Jupyter, with Git and GitHub for version control. The code forms one package, medintel, with a module per component (NFR3), and every path, version, weight and threshold sits in one settings.toml file (NFR6). Listings omit constructors that only store their arguments; the data objects are dataclasses in medintel/models.py.

5.2 CORE PIPELINE AND ORCHESTRATION :

5.2.1 PROJECT INITIALIZATION AND DATA LOADING :

Initialization reads settings.toml and builds the components in pipeline order, starting with data loading (FR1). DataIngestionService reads a CSV file with pandas, drops exact duplicates and turns empty cells into None. SchemaValidator then checks each row against a pydantic model, to which unit checks will be added once the dataset is fixed. Invalid rows are set aside for the Data Administrator.

Python

```
# medintel/ingestion.py (data loading and schema validation)
import pandas as pd
from pydantic import BaseModel, ValidationError
from medintel.models import PatientCase

class RecordSchema(BaseModel):
    patient_id: str
    age: int
    heart_rate: float | None = None
```

```
    clinical_notes: str | None = None

class SchemaValidator:
    def validate(self, record: dict) -> list[str]:
        try:
            RecordSchema.model_validate(record)
        except ValidationError as err:
            return [f"{e['loc'][0]}: {e['msg']}" for e in err.errors()]
        return []

class DataIngestionService:
    def load(self, path: str) -> list[PatientCase]:
        frame = pd.read_csv(path, dtype={"patient_id": str}).drop_duplicates()
        frame = frame.astype(object).where(frame.notna(), None)  # empty cells become None
        cases = []
        for row_no, record in frame.to_dict("index").items():
            if problems := self.validator.validate(record):
                self.rejected.append((row_no, problems))
            else:
                cases.append(PatientCase.from_record(record, source=f"{path}:{row_no}"))
        return cases
```

The assignment expression in the if statement keeps malformed rows out of the pipeline without raising an exception (NFR5); PatientCase.from_record then fills x_s, x_t and the source metadata, and event logs join as x_l by patient identifier.

5.2.2 IMPLEMENTATION OF THE ORCHESTRATOR AND CASE QUEUE :

The Orchestrator runs the pipeline for one case and the CaseScheduler runs many cases at once (FR2). Since the machine-learning, rule and retrieval agents are independent, analyze() runs them in a three-thread pool, and synthesis, fusion and report building wait for all three outputs (Section 4.2.3). Every report, even a failure report, is audited before it is returned.

Python

```
# medintel/orchestrator.py (orchestrator and parallel case queue)
import copy
from concurrent.futures import ThreadPoolExecutor
from medintel.models import DecisionReport, PatientCase

class Orchestrator:
    def analyze(self, case: PatientCase) -> DecisionReport:
        calls = (self.ml_agent.analyze, self.rule_agent.evaluate, self.retriever.retrieve)
        try:
            with ThreadPoolExecutor(max_workers=3) as pool:
                ml, rules, evidence = pool.map(lambda call: call(case), calls)
            context = self.context_agent.synthesize(case, ml, rules, evidence)
            fusion = self.fusion_engine.fuse(ml, rules, evidence, context)
            report = self.explainer.build_report(case, fusion)
        except Exception as exc:  # reported, never raised (NFR5)
            report = DecisionReport.failed(case, getattr(exc, "category", "unknown"))
        self.audit_logger.log(report)
        return report
```

```
class CaseScheduler:
    def submit(self, cases: list[PatientCase]) -> None:
        self.queue.extend(copy.deepcopy(case) for case in cases)  # isolated copies (NFR2)

    def run(self, max_workers: int) -> list[DecisionReport]:
        batch, self.queue = self.queue, []
        with ThreadPoolExecutor(max_workers=max_workers) as pool:
            return list(pool.map(self.orchestrator.analyze, batch))
```

In submit(), the deep copy keeps each patient's data separate (NFR2), and in analyze() the except branch turns any agent error into a categorized failure report instead of stopping the batch (NFR5).

5.3 CODING THE ANALYSIS AGENTS :

5.3.1 MACHINE-LEARNING ANALYSIS AGENT :

MLAnalysisAgent wraps a scikit-learn Pipeline that is to be trained offline in a notebook and saved with joblib (FR3). The pipeline imputes and scales features before its final step, which may be logistic regression, a random forest or an XGBoost classifier, so a model change leaves the agent untouched. Feature importance is read once, at load time.

Python

```
# medintel/agents/ml_agent.py (machine-learning analysis agent)
import joblib
import pandas as pd
from medintel.models import MLResult, PatientCase

class MLAnalysisAgent:
    def __init__(self, model_path: str, model_version: str) -> None:
        self.model = joblib.load(model_path)  # fitted scikit-learn Pipeline
        self.model_version = model_version
        self.feature_names = list(self.model.feature_names_in_)
        final_step = self.model[-1]
        weights = getattr(final_step, "feature_importances_", None)
        if weights is None:  # logistic regression exposes coefficients instead
            weights = abs(final_step.coef_[0])
        self.importance = dict(zip(self.feature_names, weights.round(4).tolist()))

    def analyze(self, case: PatientCase) -> MLResult:
        row = pd.DataFrame([case.x_s], columns=self.feature_names, dtype=float)
        probs = self.model.predict_proba(row)[0]
        labels = [str(label) for label in self.model.classes_]
        best = int(probs.argmax())
        return MLResult(prediction=labels[best],
                        probabilities=dict(zip(labels, probs.round(4).tolist())),
                        importance=self.importance,
                        model_version=self.model_version)
```

The probability that predict_proba assigns to the predicted class becomes the ML support M of Equation (4.2), and columns=self.feature_names keeps the training column order.

5.3.2 RULE-BASED VALIDATION AGENT :

RuleValidationAgent applies the JSON rulebase of Section 4.4.1 (FR4). RuleBase.load turns each entry into a Rule with an identifier, field, operator, threshold, action, severity and message. A rule fires when its field crosses the threshold or, for the missing operator, when the field is empty. All fired rules are reported, but the most severe one sets the action.

Python

```
# medintel/agents/rule_agent.py (rule-based validation agent)
import json
import operator
from medintel.models import PatientCase, Rule, RuleResult

OPS = {">": operator.gt, ">=": operator.ge, "<": operator.lt, "<=": operator.le}
SEVERITY = {"low": 1, "medium": 2, "high": 3}

class RuleBase:
    @classmethod
    def load(cls, path: str) -> "RuleBase":
        with open(path, encoding="utf-8") as fh:
            data = json.load(fh)
        return cls(data["version"], [Rule(**item) for item in data["rules"]])

def fires(rule: Rule, value: float | None) -> bool:
    if rule.operator == "missing":
        return value is None
    return value is not None and OPS[rule.operator](value, rule.threshold)

class RuleValidationAgent:
    def evaluate(self, case: PatientCase) -> RuleResult:
        hits = [r for r in self.rulebase.rules if fires(r, case.x_s.get(r.field))]
        top = max(hits, key=lambda r: SEVERITY[r.severity], default=None)
        return RuleResult(triggered=[r.rule_id for r in hits],
                          flags=[r.message for r in hits],
                          action=top.action if top else None,
                          reason=top.message if top else "no rule triggered",
                          rulebase_version=self.rulebase.version)
```

Because fires() is a plain table lookup, a given case and rulebase always yield the same RuleResult, and the stored rulebase version traces every flag to its rule set (NFR1).

5.4 KEY SYSTEM IMPLEMENTATION :

5.4.1 KNOWLEDGE RETRIEVAL AGENT :

The Knowledge Retrieval Agent gathers evidence for each case (FR5), following the evidence-grounding approach of retrieval-augmented systems [6], [8], [9]. KnowledgeStore keeps guideline excerpts, articles and approved references in a FAISS inner-product index; with normalized sentence-transformers embeddings, every score is a cosine similarity. One query is formed from the case notes and feature values.

Python

```
# medintel/agents/retrieval_agent.py (knowledge retrieval agent)
import faiss
import numpy as np
from sentence_transformers import SentenceTransformer
from medintel.models import EvidenceItem, PatientCase

class KnowledgeStore:
    def __init__(self, encoder: SentenceTransformer) -> None:
        self.encoder = encoder
        self.docs: list[dict] = []
        self.index = faiss.IndexFlatIP(encoder.get_sentence_embedding_dimension())

    def search(self, vector: np.ndarray, k: int) -> list[EvidenceItem]:
        scores, ids = self.index.search(vector.reshape(1, -1), k)
        return [EvidenceItem(self.docs[i]["source_id"], self.docs[i]["passage"], float(s))
                for s, i in zip(scores[0], ids[0]) if i != -1]

    def add_validated(self, items: list[dict], approved_by: str) -> None:
        texts = [item["passage"] for item in items]
        self.index.add(self.encoder.encode(texts, normalize_embeddings=True))
        self.docs += [dict(item, approved_by=approved_by) for item in items]

class KnowledgeRetrievalAgent:
    def retrieve(self, case: PatientCase, k: int = 5) -> list[EvidenceItem]:
        query = case.x_t + " " + " ".join(f"{n} {v}" for n, v in case.x_s.items())
        vector = self.store.encoder.encode(query, normalize_embeddings=True)
        return [e for e in self.store.search(vector, k) if e.score >= self.min_score]
```

Filtering by min_score makes an empty list mean that no sufficiently relevant evidence exists, while add_validated, the only write path into the index, records the approving curator, a safeguard for the governance problem highlighted in [10].

5.4.2 DECISION FUSION AND EXPLANATION GENERATION :

Before fusion, the Clinical Context Agent (FR6) sends the ML, rule and retrieval outputs to the LLM Service in one structured message, with an instruction to use only the supplied facts. It returns a ContextSummary with the draft text, the conflicts found and any

unsupported statements; if the service cannot be reached, the summary records a synthesis error and the other evidence still reaches the report (assumption A3). DecisionFusionEngine (FR7) then evaluates Equation (4.2) for the planned binary risk task. M is the probability of the predicted class and R drops to zero when a rule overrides the model; K is the mean similarity of the kept passages, and D is the number of reported conflicts divided by the three evidence sources, capped at 1.

Python

```
# medintel/fusion.py (decision fusion and explanation generation)
from medintel.models import (ContextSummary, DecisionReport, EvidenceItem, FusionResult,
                             MLResult, PatientCase, RuleResult)

OVERRIDES = {"elevate": "high_risk", "suppress": "withheld"}

class DecisionFusionEngine:
    def fuse(self, ml: MLResult, rules: RuleResult, evidence: list[EvidenceItem],
             context: ContextSummary) -> FusionResult:
        override = OVERRIDES.get(rules.action)
        m = ml.probabilities[ml.prediction]
        r = 0.0 if override and override != ml.prediction else 1.0
        k = sum(e.score for e in evidence) / len(evidence) if evidence else 0.0
        d = min(1.0, len(context.conflicts) / 3)  # three sources, at most three conflicts
        c = self.alpha * m + self.beta * r + self.gamma * k - self.lam * d
        decision = override or (ml.prediction if c >= self.threshold else "uncertain")
        return FusionResult(decision, c, d, {"M": m, "R": r, "K": k},
                            ml, rules, evidence, context)

class ExplanationGenerator:
    def build_report(self, case: PatientCase, fusion: FusionResult) -> DecisionReport:
        summary = fusion.context.summary
        for claim in fusion.context.unsupported_statements:
            summary = summary.replace(claim, "")
        weights = fusion.ml.importance
        top = sorted(weights, key=weights.get, reverse=True)[:3]
        findings = [f"{name} = {case.x_s.get(name)}" for name in top] + fusion.rules.flags
        return DecisionReport.create(case, fusion, findings, summary.strip())
```

The line computing c is Equation (4.2), with lam for λ since lambda is reserved in Python; its weights and threshold are to be tuned only on the validation split. A rule override always wins, a score below the threshold yields uncertain, and unsupported statements are removed before DecisionReport.create fills the Section 4.4.2 fields (FR8).

5.5 CODE SNIPPETS AND DOCUMENTATION :

5.5.1 CRITICAL PYTHON FUNCTIONS (AUDIT LOGGING AND REPORT EXPORT) :

Two functions complete the audit trail (FR8, NFR1). AuditLogger.log stores one SQLite row per report with the case identifier, model and rulebase versions, retrieved source identifiers, status and a UTC timestamp, and runs again after every review, so each status change keeps its own row. The export_report function saves the report as JSON in the Section 4.4.2 format.

Python

```
# medintel/audit.py (audit logging and report export)
import json
import sqlite3
import threading
from dataclasses import asdict, astuple
from datetime import datetime, timezone
from pathlib import Path
from medintel.models import AuditRecord, DecisionReport

SCHEMA = ("CREATE TABLE IF NOT EXISTS audit (case_id TEXT, model_version TEXT, "
        "rulebase_version TEXT, retrieval_ids TEXT, status TEXT, timestamp TEXT)")

class AuditLogger:
    def __init__(self, db_path: str) -> None:
        self.lock = threading.Lock()
        self.db = sqlite3.connect(db_path, check_same_thread=False)
        self.db.execute(SCHEMA)

    def log(self, report: DecisionReport) -> AuditRecord:
        record = AuditRecord(report.case_id, report.versions["model"],
                             report.versions["rulebase"], ",".join(report.evidence_refs),
                             report.status, datetime.now(timezone.utc).isoformat())
        with self.lock, self.db:  # one writer at a time, committed on exit
            self.db.execute("INSERT INTO audit VALUES (?,?,?,?,?,?)", astuple(record))
        return record

def export_report(report: DecisionReport, out_dir: str) -> Path:
    path = Path(out_dir) / f"{report.case_id}.json"
    path.write_text(json.dumps(asdict(report), indent=2), encoding="utf-8")
    return path
```

Writes go through a lock because the scheduler’s worker threads share one SQLite connection; the PostgreSQL store planned for deployment keeps the same AuditRecord fields.

5.5.2 IMPLEMENTATION OF THE CLINICIAN REVIEW INTERFACE :

ClinicianDashboard is the Streamlit page on which the Clinician reviews a report and records a decision (FR9). It reaches the backend only through the REST endpoints of Section 3.3.3, using the requests library that Streamlit already installs. The show_report method keeps the field order of Section 3.3.1, from salient findings to confidence and recommendation, and shows each conflict as a warning (NFR7).

Python

```
# medintel/dashboard.py (clinician review interface)
import requests
import streamlit as st
from medintel.models import DecisionReport

class ClinicianDashboard:
    def show_report(self, report: DecisionReport) -> None:
        st.subheader(f"Case {report.case_id}, patient {report.patient_id}")
        st.write("Salient findings:", report.findings)
        st.write("Model prediction:", report.prediction)
        st.write("Triggered rules:", report.triggered_rules)
        st.write("Evidence references:", report.evidence_refs)
        for conflict in report.conflicts:
            st.warning(f"Conflict: {conflict}")
        st.metric("Confidence C", f"{report.confidence:.2f}")
        st.info(report.recommendation)

    def submit_review(self, case_id: str, decision: str, notes: str) -> None:
        reply = requests.post(f"{self.api_url}/cases/{case_id}/review",
                              json={"decision": decision, "notes": notes}, timeout=10)
        reply.raise_for_status()

    def render(self) -> None:
        case_id = st.selectbox("Case", self.case_ids)
        reply = requests.get(f"{self.api_url}/cases/{case_id}/report", timeout=10)
        self.show_report(DecisionReport(**reply.json()))
        decision = st.radio("Decision", ["accepted", "rejected", "annotated"])
        notes = st.text_area("Clinician notes")
        if st.button("Submit review"):
            self.submit_review(case_id, decision, notes)
```

The submit_review method only posts the decision and notes. Behind POST /cases/{case_id}/review, the backend sets the new status, writes an audit record and saves the report, but it never writes to the knowledge store: accepted cases wait for the Knowledge Curator’s approval through add_validated, as the loop in Fig 4.2.3 requires.

SUMMARY :

Chapter 5 described how the MedIntel design is to be coded in Stage II. After the tools and package layout, it gave reference code for every module in pipeline order. The listings cover

schema-checked loading, parallel scheduling with isolated case copies, the scikit-learn and JSON rule agents, FAISS retrieval with a curator-only write path and the confidence function of Equation (4.2). They end with SQLite audit logging and a Streamlit review page that works through the REST interface. The Stage II test plan will build on these interfaces.

CHAPTER 06

CONCLUSION

This chapter closes the Stage I report on MedIntel in numbers. It lists what the design phase has delivered and the Stage II targets it has fixed, quantifies the four contributions that set the framework apart from the systems reviewed in Chapter 2, and records the six challenges met while designing the pipeline, together with the mitigation planned for each.

6.1 CONCLUSION OF THE PROJECT :

Stage I has turned the project idea into a complete, testable specification. Table 6.1 summarizes its deliverables in numbers, with the place in the report where each one is defined.

Table 6.1 : Stage I Deliverables In Numbers

Sr. No.	Stage I Deliverable	Count / Value	Reference
1	Published works reviewed and compared	12	Table 2.2
2	Research gaps identified	2	Section 2.6
3	Design constraints carried through the architecture	4	Section 4.1
4	User classes	4	Table 3.1.2
5	Assumptions and technical dependencies	5 (A1 to A5)	Section 3.1.3
6	Functional requirements	9 (FR1 to FR9)	Table 3.2
7	Non-functional requirements	7 (NFR1 to NFR7)	Table 3.5
8	REST endpoints in the backend interface	4	Section 3.3.3
9	Components with typed output contracts	7	Table 4.2.1
10	External entities in the Level 0 DFD	5	Fig 4.1.2 (a)
11	UML diagrams (use case, class and two sequence diagrams)	4	Section 4.3
12	Lifelines modelled in the sequence diagram	13	Section 4.3.3
13	Evidence terms in the confidence function (M, R, K, D)	4	Equation (4.2)
14	Planned evaluation conditions, from ML alone to the full pipeline	4	Section 1.4
15	Plagiarism similarity of the checked chapters	0%, 2% and 0%	Appendix C
16	Research paper submitted (IJAICMES)	1	Appendices B and D

The same specification fixes the numerical targets that the Stage II prototype must meet:

- End-to-end processing time of at most 30 seconds per case on the recommended hardware (NFR4).
- ML and rule stages under 1 second each (NFR4).
- A complete AuditRecord for 100% of reports (NFR1).
- A report that a clinician can read and understand in under 2 minutes (NFR7).
- Every failure logged in 1 of 6 categories, with no crash on malformed or missing data (NFR5).
- Hardware of 8 GB RAM and 20 GB free SSD space as the minimum, and 16 GB RAM and 50 GB as the recommended configuration (Table 3.4).

No accuracy or clinical-performance figure is claimed at this stage. Every performance number above is a target that Stage II will measure.

6.2 SUMMARY OF CONTRIBUTION AND UNIQUENESS :

MedIntel's contribution lies in system integration, traceability and reproducibility rather than in new algorithms. Table 6.2 quantifies the four elements that make it different from the systems reviewed in Chapter 2.

Table 6.2 : Contributions Of MedIntel In Numbers

Sr. No.	Contribution	Quantified Design Element
1	Evidence separation with explicit output contracts	3 evidence agents (ML, rules, retrieval), each returning a typed result with a model version, rulebase version or source identifier; 7 components with typed output contracts in total
2	Explicit confidence function with a disagreement penalty	1 equation, $C = \alpha M + \beta R + \gamma K - \lambda D$ (Equation 4.2), with 4 evidence terms and 4 weights tuned only on a separate validation set
3	Explanation and audit layer	1 DecisionReport with 8 fields per case and 1 append-only AuditRecord with 5 fields (model version, timestamp, rule version, retrieval ids, final status)
4	Patient-isolated parallel execution with governed knowledge updates	1 isolated context per case and 1 write path into the knowledge store (add_validated), used only after 2 approvals: clinician review and Knowledge Curator approval

6.3 CHALLENGES FACED AND MITIGATION STRATEGIES :

Because MedIntel is at the design stage, the challenges below are those met while designing the pipeline and expected to matter most during implementation. Each of the 6 mitigations is

a decision already built into the architecture, and whether it works will be checked in Stage II against the requirements of Chapter 3, in particular the NFR targets of Table 3.5. Table 6.3 summarizes the six challenges and the planned mitigation for each.

Table 6.3 : Challenges Faced And Mitigation Strategies

Challenge	Description	Mitigation Strategy
Heterogeneous And Incomplete Patient Data	Records arrive as CSV files, tables and event logs with missing fields, inconsistent units, repeated observations and free-text notes that describe one episode in different ways.	The SchemaValidator checks types, missing fields, duplicates, units and identifiers before a PatientCase is built; rules flag missing required fields; failures are logged by category without stopping the batch (NFR5).
Conflicts Between ML And Rules	The model prediction can disagree with a deterministic rule, for example a low predicted risk while a laboratory value is beyond its threshold.	A rule may elevate, suppress or flag a recommendation but must record its reason; the ClinicalContextAgent reports the conflict explicitly, and the λD term lowers the confidence score.
LLM Hallucination Risk	A generative model may state findings or evidence that the patient data and the retrieved sources do not support.	The LLM runs after the rules and only synthesizes structured agent outputs; it must not invent observations and must say when evidence is missing, and any unsupported statement is listed in the ContextSummary and kept out of the findings.
Knowledge-Store Contamination	If model outputs were re-indexed automatically, one incorrect recommendation could become a persistent source of future errors.	KnowledgeStore.add_validated is the only write path: only clinician-validated outcomes approved by the Knowledge Curator are re-indexed, with source identifiers and a versioned index.
Computational Cost And Latency	Several agents need more inference calls, retrieval steps and serialization overhead than a single predictive model, which raises the time and cost of each case.	Independent agents run in parallel, generative synthesis is kept to the final stage, preprocessing stays deterministic where possible, and latency and per-case cost will be measured against the NFR4 target.

Challenge	Description	Mitigation Strategy
Patient Privacy	Clinical records are sensitive, many patients are handled in one batch, and synthesis may send case content to an external LLM Service.	The Data Administrator de-identifies records before analysis, each case runs in an isolated context (NFR2), and the configurable LLM endpoint allows a locally served model so that case content stays on premises.

SUMMARY :

Chapter 6 concluded the Stage I work in numbers: 12 works reviewed, 2 research gaps, 9 functional and 7 non-functional requirements, 7 components with typed output contracts, a confidence function with 4 evidence terms, and 6 design-stage challenges with planned mitigations. No performance is claimed at this stage; Stage II will implement the framework and measure it against the targets listed in Section 6.1.

REFERENCES

[1] Ç. U. Öğdü, K. Arslanoğlu, and M. Karaköse, "*An Adaptive Multi-Agent LLM-Based Clinical Decision Support System Integrating Biomedical RAG and Web Intelligence*," IEEE Access, vol. 13, pp. 167390-167404, 2025, doi: 10.1109/ACCESS.2025.3613340.

[2] X. Tang, A. Zou, Z. Zhang, Z. Li, Y. Zhao, X. Zhang, A. Cohan, and M. Gerstein, "*MedAgents: Large Language Models as Collaborators for Zero-shot Medical Reasoning*," in Findings of the Association for Computational Linguistics: ACL 2024, pp. 599-621, 2024, doi: 10.18653/v1/2024.findings-acl.33.

[3] Y. Lu, X. Zhao, and J. Wang, "*ClinicalRAG: Enhancing Clinical Decision Support through Heterogeneous Knowledge Retrieval*," in Proc. 1st Workshop on Towards Knowledgeable Language Models (KnowLLM 2024), pp. 64-68, 2024, doi: 10.18653/v1/2024.knowllm-1.6.

[4] K. Zuo, Y. Jiang, F. Mo, and P. Lio, "*KG4Diagnosis: A Hierarchical Multi-Agent LLM Framework with Knowledge Graph Enhancement for Medical Diagnosis*," in Proc. First AAAI Bridge Program on AI for Medicine and Healthcare, PMLR, vol. 281, pp. 195-204, 2025.

[5] T. Abdullahi, L. Mercurio, R. Singh, and C. Eickhoff, "*Retrieval-Based Diagnostic Decision Support: Mixed Methods Study*," JMIR Medical Informatics, vol. 12, e50209, 2024, doi: 10.2196/50209.

[6] S. Liu, A. B. McCoy, and A. Wright, "*Improving large language model applications in biomedicine with retrieval-augmented generation: a systematic review, meta-analysis, and clinical development guidelines*," Journal of the American Medical Informatics Association, vol. 32, no. 4, pp. 605-615, 2025, doi: 10.1093/jamia/ocaf008.

[7] S. Delourme, A. Redjdal, J. Bouaud, and B. Seroussi, "*Measured Performance and Healthcare Professional Perception of Large Language Models Used as Clinical Decision Support Systems: A Scoping Review*," Studies in Health Technology and Informatics, vol. 316, pp. 841-845, 2024, doi: 10.3233/SHTI240543.

[8] C. Wang, J. Ong, C. Wang, H. Ong, R. Cheng, and D. Ong, "*Potential for GPT Technology to Optimize Future Clinical Decision-Making Using Retrieval-Augmented Generation*," Annals of Biomedical Engineering, vol. 52, no. 5, pp. 1115-1118, 2024, doi: 10.1007/s10439-023-03327-6.

[9] S. Kresevic, M. Giuffrè, M. Ajcevic, A. Accardo, L. S. Crocè, and D. L. Shung, "*Optimization of hepatological clinical guidelines interpretation by large language models: a retrieval augmented generation-based framework*," npj Digital Medicine, vol. 7, art. no. 102, 2024, doi: 10.1038/s41746-024-01091-y.

[10] W. Li, R. Yan, X. Zhang, L. Chen, H. Zhu, J. Zhao, J. Li, M. Li, W. Cao, Z. Jiang, W. Wei, K. Zhang, and S. K. Zhou, "*MACD: Multi-Agent Clinical Diagnosis with Self-Learned Knowledge for LLM*," arXiv preprint arXiv:2509.20067, 2025.

[11] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, "*Large language models in medicine*," Nature Medicine, vol. 29, no. 8, pp. 1930-1940, 2023, doi: 10.1038/s41591-023-02448-8.

[12] A. Patel, Y. Khand, S. K. Vallamchetla, P. Li, C. Tao, and J. Cheung, "*Fine-Tuning, Retrieval-Augmented Generation, and Hybrid Adaptation of Language Models for Clinical Decision-Making in Health Care: Systematic Review*," Journal of Medical Internet Research, vol. 28, e104092, 2026, doi: 10.2196/104092.

APPENDIX A : BASE PAPER

The two base papers of this project, listed in Table A.1, are the adaptive multi-agent clinical decision support system of Öğdü et al. [1] and the MedAgents framework of Tang et al. [2]; the datasets available from them, together with sample datasets prepared by the team, are the intended starting point for the Stage II experiments.

Table A.1 : Base Paper Details

Field	Base Paper 1	Base Paper 2
Title	An Adaptive Multi-Agent LLM-Based Clinical Decision Support System Integrating Biomedical RAG and Web Intelligence	MedAgents: Large Language Models as Collaborators for Zero-shot Medical Reasoning
Authors	Ç. U. Öğdü, K. Arslanoğlu, and M. Karaköse	X. Tang, A. Zou, Z. Zhang, Z. Li, Y. Zhao, X. Zhang, A. Cohan, and M. Gerstein
Published In	*IEEE Access*, vol. 13, pp. 167390-167404	*Findings of the Association for Computational Linguistics: ACL 2024*, pp. 599-621
Year	2025	2024
DOI	10.1109/ACCESS.2025.3613340	10.18653/v1/2024.findings-acl.33

Base Paper 1: Öğdü et al. [1] propose an adaptive multi-agent clinical decision support system based on large language models (LLMs). Its architecture couples specialist agents with biomedical retrieval-augmented generation (RAG), web evidence, data fusion and a confidence-triggered feedback loop, and it is closely related to MedIntel in its emphasis on modularity and confidence-aware processing. Together with other multi-agent systems, it demonstrates the value of decomposition and collaboration, yet it also illustrates an engineering trade-off: the more reasoning components a system contains, the harder it becomes to identify which of them produced a particular recommendation.

Base Paper 2: Tang et al. [2] present MedAgents, in which role-specific large language model agents act as collaborators for zero-shot medical reasoning. The agents first propose independent analyses of a question, then discuss their findings, summarize the discussion and reach a decision. The study shows why decomposition can help medical reasoning: each agent can concentrate on a distinct perspective before the final synthesis. The modularity reported in MedAgents is one of the influences on the MedIntel design.

MedIntel builds on both papers but changes where the analysis takes place. Like MedAgents [2], it assigns tasks to specialized agents, yet it moves the analytical work beyond language-model agents by adding dedicated machine-learning and deterministic rule components. Compared with the adaptive system of Öğdü et al. [1], MedIntel is deliberately narrower: the learned prediction and the rule engine sit at the center of patient-level analysis of structured data, while retrieval and language generation act as supporting layers. Instead of treating multi-agent behavior as unrestricted conversation, MedIntel gives every component a clear output contract, so that the origin of each recommendation can be traced. Its novelty therefore lies less in new algorithms than in system integration, traceability and reproducibility.

APPENDIX B : IEEE PAPER

MedIntel: A Multi-Agent AI Framework for Clinical Decision Support Using Machine Learning, Rule-Based Validation, and Knowledge Retrieval

G. P. Mohole
HOD, Computer Engineering
JESITMR, Nashik, India
gpmohole@jitnashik.edu.in

Siddhesh Suryawanshi
Department of Computer Engineering
JESITMR, Nashik, India
siddheshksuryawanshi@gmail.com

Shantanu Patil
Department of Computer Engineering
JESITMR, Nashik, India
shantanupatil1155@gmail.com

Omkar Musale
Department of Computer Engineering
JESITMR, Nashik, India
omkar98musale@gmail.com

Tanmay Bagul
Department of Computer Engineering
JESITMR, Nashik, India
tanmaybagul48@gmail.com

Abstract— Clinical decision support systems increasingly combine machine learning, knowledge retrieval, and language-model reasoning, yet the integration of these components can become difficult to audit when a single generative model is responsible for the final recommendation. This paper presents MedIntel, a proposed multi-agent framework that treats patient analysis as a coordinated pipeline rather than a single prediction step. Its objective is an implementable, modular architecture for patient-level decision support in which every recommendation can be traced to the evidence that produced it. The framework accepts structured patient records and associated event or log data, validates and normalizes the input, applies task-specific machine-learning analysis, executes deterministic rule checks, retrieves supporting medical knowledge, and then combines the resulting evidence through a decision-fusion layer whose explicit confidence function weighs model, rule, and retrieval support against disagreement among components. A dedicated explanation and audit stage converts technical outputs into a structured report that can be reviewed by a clinician. The design also supports parallel processing of multiple patient records in isolated contexts and controlled re-indexing of validated knowledge. Unlike systems that rely mainly on free-form large language model (LLM) inference, MedIntel assigns explicit responsibilities to analytical components and reserves generative models for contextual synthesis and explanation. The present work is a design-stage study; therefore, no empirical accuracy or clinical-performance claims are made. Instead, an evaluation protocol is defined for future validation using benchmark datasets, progressively richer baselines, component ablation studies, calibration measures, and human review of explanation quality. By separating learned, rule-based, and retrieved evidence, the framework aims to make patient-level clinical reasoning more traceable and easier to audit.

Keywords— clinical decision support, multi-agent systems, retrieval-augmented generation, rule-based reasoning, explainable AI, healthcare AI

I. INTRODUCTION

MedIntel is a medical log analysis system for generating useful diagnostic results. Clinical decision support systems (CDSSs) have traditionally been built around rules, statistical models, or retrieval services that answer narrowly defined clinical questions [1]. MedAgents uses several role-based large language model (LLM) agents that independently analyze a question, discuss the findings, and reach a consensus [2]. ClinicalRAG similarly uses a multi-agent pipeline to connect clinical inputs with heterogeneous structured and unstructured knowledge [3]. KG4Diagnosis extends the idea by combining hierarchical agents with a medical knowledge graph for specialized diagnosis [4]. These systems can be useful because their inputs and outputs are comparatively structured, yet real hospital data rarely arrives in a perfectly normalized form. An ensemble combining complementary retrieval and predictive approaches can improve diagnostic ranking, including for less common conditions [5], [6]. Patient records may contain missing fields, inconsistent units, repeated observations, free-text notes, and event logs that describe the same clinical episode from different perspectives. The challenge is therefore not only to predict a condition, but also to decide which pieces of evidence should influence that prediction and how the resulting recommendation can be explained to a human reviewer.

The rapid development of LLMs has expanded the design space. Modern reviews of clinical LLM systems identify calibration, validation, and transparency as recurring weaknesses [7]. These weaknesses can also be seen broadly in retrieval-augmented generation (RAG) studies [8], [9]. Research on self-learned clinical knowledge has made the governance problem more visible [10]. A review by Thirunavukarasu et al. describes broad applications of LLMs in medicine while emphasizing the domain-specific risks and validation requirements that accompany clinical use [11]. Subsequent studies have shown that language models can support clinical decision tasks [12], especially when external knowledge is retrieved at inference time. Wang et al. discussed RAG as a mechanism for making model responses more specific to current clinical evidence [8], while Kresevic et al. demonstrated that structured guideline retrieval and careful prompt construction can improve the accuracy of guideline interpretation [9].

More recent work has moved from single-model prompting toward multi-agent systems. In 2025, Öğdü et al. proposed an adaptive architecture that couples specialist agents, biomedical RAG, web evidence, data fusion, and a confidence-triggered feedback loop [1]. These works demonstrate the value of decomposition and collaboration, but they also illustrate an important engineering trade-off: the more reasoning components are introduced, the more difficult it becomes to identify which component produced a particular recommendation.

MedIntel is motivated by a simpler principle: different kinds of evidence should be handled by the type of component that is best suited to that evidence. Machine-learning models are appropriate for learned patterns in structured features; rule engines are appropriate for explicit thresholds and institutional policies; retrieval systems are appropriate for contextual evidence; and language models are useful for synthesis and explanation. Rather

than asking one model to perform every task, the proposed framework coordinates these components and preserves their outputs as an auditable evidence set.

The principal objective of this work is therefore to define an implementable, modular architecture for patient-level decision support that can process multiple patient records in parallel, combine learned and deterministic evidence, produce structured explanations, and preserve a trace of the inputs that led to the recommendation. Because the work is currently at the proposal stage, this paper is intentionally positioned as a design and methodology study. No experimental results are reported; instead, the paper specifies how the proposed system should be evaluated once implementation is complete.

II. LITERATURE REVIEW

Recent research can be organized around four complementary directions: LLM-based medical reasoning, retrieval-augmented clinical systems, multi-agent diagnostic frameworks, and predictive or rule-based decision support. The broader medical-LLM literature recognizes the promise of language models for clinical tasks while emphasizing risks related to factuality, context dependence, and clinical validation [11]. These limitations motivate the use of explicit evidence sources rather than unrestricted model memory.

A. LLM-Based Medical Reasoning MedAgents is a representative multi-agent approach in which role-specific language-model agents propose independent analyses, discuss the findings, summarize the discussion, and reach a decision [2]. The study shows why decomposition can help medical reasoning: different agents can focus on distinct perspectives before a final synthesis. MedIntel adopts the same general principle of specialization but moves the analytical responsibilities beyond LLMs by introducing dedicated machine-learning and deterministic-rule components.

B. Retrieval-Grounded Clinical Systems RAG-based approaches attempt to connect language-model outputs to external evidence. Wang et al. argued that retrieval can make clinical responses more specific to current medical information [8]. Kresevic et al. subsequently showed how structured guideline retrieval and careful prompt construction can substantially improve guideline interpretation for a focused clinical task [9]. Liu et al. conducted a systematic review and meta-analysis of biomedical RAG studies and reported an overall performance benefit over baseline LLMs, while also identifying methodological weaknesses that warrant stronger validation [6]. ClinicalRAG extended this direction through heterogeneous structured and unstructured medical knowledge in a multi-agent pipeline [3].

C. Knowledge-Structured and Hierarchical Agents KG4Diagnosis combines a general practitioner-style agent with specialized agents and an automatically constructed medical knowledge graph [4]. This provides a useful model for representing relationships between symptoms and diseases, but it also introduces additional knowledge-engineering requirements. The 2025 adaptive multi-agent CDSS by Öğdü et al. integrates biomedical RAG, web evidence, data fusion, and an adaptive feedback loop [1]. Their architecture is closely related to MedIntel in its emphasis on modularity and confidence-aware processing, but MedIntel is deliberately narrower: it places the learned prediction and rule engine at the center of patient-level structured-data analysis and uses retrieval and language generation as supporting layers.

D. Retrieval-Based Diagnosis and the Remaining Gap Abdullahi et al. found that an ensemble combining complementary retrieval and predictive approaches can improve diagnostic ranking, including for less common conditions [5]. Taken together, these studies suggest that a robust clinical-support pipeline benefits from multiple evidence sources, yet they do not provide a simple, auditable architecture in which tabular machine learning, deterministic rules, retrieval, and generative synthesis share a common patient-level interface. MedIntel addresses this engineering gap by defining clear contracts between these components rather than treating multi-agent behavior as unrestricted conversation. The resulting design is intentionally modest in algorithmic novelty and stronger in system integration, traceability, and reproducibility.

III. METHODOLOGY

This section formalizes the problem, describes the proposed MedIntel architecture, and defines the protocol for its implementation and evaluation.

A. Problem Formulation and Design Objectives Let a patient case be represented as a heterogeneous input $X = \{x_s, x_t, x_l\}$, where x_s denotes structured attributes such as age, laboratory values, vital signs, or categorical features; x_t denotes text-based clinical context; and x_l denotes related event or log observations. The system produces a candidate decision y, an evidence set E, and an explanation z. MedIntel seeks to construct a function F such that

$$(y, E, z) = F(X, R, K), \tag{1}$$

where R is a deterministic rulebase and K is an external or indexed knowledge source.

The architecture is guided by five objectives. First, it should separate learned inference from deterministic validation. Second, it should maintain provenance so that each important recommendation can be traced to an input feature, rule, model output, or retrieved evidence item. Third, it should support batch execution across multiple patient cases without mixing patient context. Fourth, it should produce a human-readable report instead of exposing raw model outputs. Finally, it should remain modular so that an individual agent or model can be replaced without redesigning the entire pipeline. These objectives translate into four design constraints:

- *Patient isolation:* no evidence, retrieved passage, or rule result from one case may be reused in another case unless it is part of the shared knowledge base.
- *Deterministic guardrails:* explicit thresholds and policies are executed by a rule component before generative synthesis.
- *Evidence grounding:* retrieved knowledge is stored with identifiers and source metadata so that supporting evidence can be inspected.
- *Human oversight:* the system provides recommendations for review and does not act as an autonomous diagnostic authority.

B. Proposed MedIntel Architecture Fig. 1 shows the proposed MedIntel architecture. Its components are described in the following subsections.

1) Input Ingestion and Preprocessing The first stage accepts patient data from CSV files, tabular datasets, or associated logs. A schema validator checks data types, missing fields, duplicate records, unit consistency, and identifier integrity. The normalized representation is converted into a patient-specific case object so that downstream agents consume a stable interface. Text fields can undergo lightweight cleaning and entity extraction, while numeric variables are preserved in their original semantic units before model-specific preprocessing.

2) Machine-Learning Analysis Agent The Machine Learning Analysis Agent is responsible for learned patterns that can be expressed as classification, risk scoring, or anomaly detection tasks. The specific algorithm is intentionally left configurable. Depending on the dataset, the implementation may compare logistic regression, random forest, gradient boosting, or another suitable baseline. The agent returns a prediction, class probabilities when available, feature importance or explanation metadata,

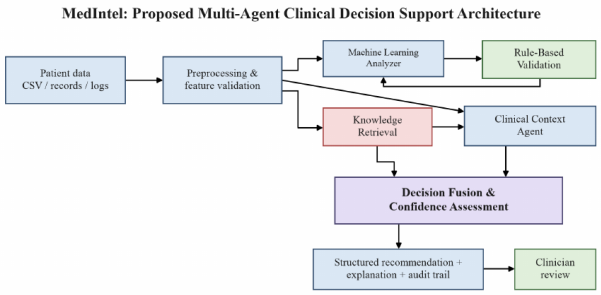


Fig. 1. Proposed MedIntel architecture combining learned inference, deterministic rule checks, retrieval-based evidence, and human review.

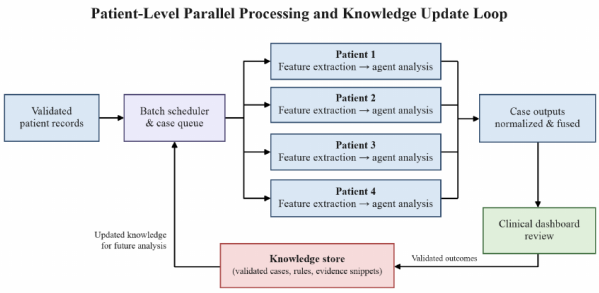


Fig. 2. Parallel case processing with isolated patient contexts and controlled knowledge-store updates.

and the model version used. This output is treated as evidence rather than as a final diagnosis.

3) Rule-Based Validation Agent The Rule-Based Validation Agent executes explicit clinical or project-defined conditions. For example, a rule can flag an input when a laboratory value exceeds a predefined threshold, when two findings occur together, or when a required field is missing. Rules are represented separately from the language model so that their behavior is deterministic and easy to audit. A rule may elevate, suppress, or flag a candidate recommendation, but it should not silently override a learned prediction without recording the reason.

4) Knowledge Retrieval Agent The Knowledge Retrieval Agent provides contextual evidence from an indexed medical knowledge collection. This design follows the broader evidence-grounding principle supported by RAG studies [6], [8], [9]. A practical implementation can store clinical articles, guideline excerpts, validated educational material, or institution-approved references in a vector index. A query derived from the current patient case is embedded and matched against the collection. Retrieved passages are returned with source identifiers and similarity scores, allowing the final response to state not only what was retrieved but also where it came from.

5) Clinical Context and Synthesis Agent The synthesis agent receives the outputs of the ML, rule, and retrieval components in a structured message. Its role is to reconcile the evidence, identify agreement or conflict, and generate a concise clinical-support report. The agent is instructed not to invent unsupported observations. Where the ML model and rule engine disagree, the conflict is reported explicitly. Where retrieval produces no sufficiently relevant evidence, the explanation should say so rather than filling the gap from unrestricted model memory.

6) Decision Fusion and Confidence Layer A conceptual confidence function can be defined as

$$C = \alpha M + \beta R + \gamma K - \lambda D, \tag{2}$$

where M is normalized machine-learning support, R is rule consistency, K is retrieval support, D represents disagreement among components, and the coefficients α, β, γ, and λ control the relative weight of each evidence source. This equation is not presented as a clinical standard; it is a design mechanism that makes the fusion logic explicit and testable. In the eventual implementation, the coefficients and thresholds should be learned or tuned only on a properly separated validation set.

7) Explanation and Audit Layer A dedicated explanation and audit stage is particularly important because modern reviews of clinical LLM systems identify calibration, validation, and transparency as recurring weaknesses [7], [12]. The final stage converts the machine-generated reasoning trace into a structured report containing: patient identifier, salient findings, model prediction, triggered rules, retrieved evidence references, conflicts, confidence indicators, and a short recommendation for clinician review. An audit record should retain the model version, timestamp, rule version, retrieval identifiers, and final status.

8) Multi-Patient Parallel Execution The system is designed to process several patient cases in parallel, as shown in Fig. 2. A batch scheduler places validated records into a queue and assigns each case an isolated context. The agents can then work concurrently where dependencies permit, while the fusion stage waits for all required evidence for a particular case. This structure makes the design suitable for hospital datasets containing many patients and also simplifies performance measurement because each case becomes an independent evaluation unit.

C. Implementation and Evaluation Protocol Since MedIntel is currently an idea-level proposal, this section defines the experimental protocol that should be followed during implementation. The primary goal is not to maximize a single accuracy number, but to determine whether each architectural component contributes measurable value and whether the complete system remains reliable under controlled conditions.

1) Dataset Preparation The datasets available from the base papers [1], [2], along with sample datasets created by the authors, are the intended starting point. A suitable public clinical dataset should be selected according to the target task and documented before implementation. The data should be split into training, validation, and held-out test partitions without patient leakage. If the project uses multiple sources, identifiers and schemas must be normalized while preserving the origin of each record. Any de-identification or synthetic-data decision should be documented.

2) Baselines and Ablation The proposed system should be compared with progressively richer baselines. A minimal baseline can use the machine-learning model alone. A second condition can add the rule engine. A third can add knowledge retrieval, while the final condition uses the complete multi-agent fusion pipeline. Ablation experiments should remove one component at a time to identify whether the system depends excessively on any single agent.

3) Evaluation Metrics Task-specific metrics should include accuracy, precision, recall, F1-score, and area under the receiver operating characteristic (ROC) curve when appropriate. Because a clinical-support system is more than a classifier, additional measurements should include calibration error, evidence retrieval precision at k, disagreement rate, false-alert rate for rules, end-to-end latency, and per-case processing cost. Human reviewers can also score explanation completeness, factual consistency, and usefulness on a defined rubric.

4) Reliability and Safety Checks The system should be evaluated on missing-data cases, contradictory evidence, borderline rule values, and deliberately ambiguous inputs [6], [12]. Each failure should be categorized as a data-validation error, model error, retrieval failure, rule conflict, synthesis error, or interface problem. This error taxonomy will make the final discussion

more informative than a single aggregate metric. Recent systematic evidence also supports emphasizing external validation and calibration rather than reporting benchmark accuracy alone.

5) Reproducibility Every experiment should record the dataset version, preprocessing configuration, model name and version, hyperparameters, rulebase version, retrieval index version, prompt template, random seeds where applicable, and evaluation script. The final implementation should be executable from a clean environment so that another researcher can reproduce the reported numbers.

IV. RESULTS AND DISCUSSION

Because MedIntel is a design-stage proposal, no experimental results are reported; this section discusses the expected strengths, costs, and limitations of the design. The main strength of MedIntel is the explicit separation of responsibilities. A machine-learning model can recognize statistical patterns without being asked to retrieve literature. A rule engine can enforce known thresholds without being asked to produce natural-language explanations. A retrieval component can ground a recommendation without assuming that retrieved text alone constitutes a diagnosis. The language model then acts as an interface between these signals, translating them into an explanation that a human can inspect.

This separation also addresses a practical weakness of highly agentic systems: when many LLM calls are chained together, errors can propagate between agents and it becomes difficult to identify where the final recommendation originated. In MedIntel, every major evidence source has a defined output contract. The fusion layer therefore has a clearer responsibility than an open-ended conversation among agents. This design is influenced by the modularity reported in MedAgents [2] and the evidence-grounding strategies explored by ClinicalRAG [3] and the 2025 adaptive CDSS study [1].

Another important consideration is computational cost. Multi-agent systems can require more inference calls, retrieval steps, and serialization overhead than a single predictive model. The proposed architecture therefore recommends parallelizing independent agents, reserving generative synthesis for the final stages, and using deterministic preprocessing whenever possible. This is consistent with the broader literature, which recognizes that improved grounding and collaboration can introduce additional latency and system complexity.

The current paper has a clear limitation: the MedIntel system has not yet been implemented, so no empirical claim about diagnostic accuracy, clinical safety, or superiority over existing methods can be made. The architecture is also intentionally generic. The most appropriate machine-learning model, retrieval strategy, rule syntax, confidence weights, and output schema will depend on the chosen dataset and clinical task.

Finally, the knowledge base should not be treated as an automatic memory that learns from every model output. Only validated and appropriately governed information should be re-indexed. This constraint is important because the knowledge-store update path can otherwise turn one incorrect recommendation into a persistent source of future errors. Recent work on self-learned clinical knowledge demonstrates the potential of iterative multi-agent learning, but it also makes the governance problem more visible [10].

V. CONCLUSION

MedIntel presents a practical architecture for multi-agent clinical decision support in which machine learning, deterministic rules, knowledge retrieval, and language-model synthesis have explicit and complementary roles. The design is intended to make patient-level reasoning more traceable while supporting parallel analysis of multiple records. Its central idea is not to replace established analytical methods with an LLM, but to orchestrate different sources of evidence and expose their contributions to human reviewers. The next step is empirical validation: implement the proposed pipeline, measure each component independently, conduct ablation and calibration studies, and evaluate the resulting system under realistic error and ambiguity conditions. Only after those experiments should performance claims be introduced into a final research manuscript.

Future work should first implement a reproducible prototype and evaluate it on a carefully selected benchmark. Subsequent work can investigate richer retrieval, medical ontologies, knowledge graphs, specialized language models, clinician-in-the-loop review, and longitudinal patient data. External validation across datasets or institutions would be particularly important because recent reviews continue to identify limited external validation and calibration as major barriers to clinical translation.

DECLARATIONS

Conflict of interest: The authors declare no conflict of interest.
Funding: This work received no external funding.
Ethical approval: Not applicable. This design-stage study did not involve human participants or animals.

REFERENCES

[1] Ç. U. Öğdü, K. Arslanoğlu, and M. Karaköse, "An Adaptive Multi-Agent LLM-Based Clinical Decision Support System Integrating Biomedical RAG and Web Intelligence," *IEEE Access*, vol. 13, pp. 167390-167404, 2025, doi: 10.1109/ACCESS.2025.3613340.

[2] X. Tang, A. Zou, Z. Zhang, Z. Li, Y. Zhao, X. Zhang, A. Cohan, and M. Gerstein, "MedAgents: Large Language Models as Collaborators for Zero-shot Medical Reasoning," in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 599-621, 2024, doi: 10.18653/v1/2024.findings-acl.33.

[3] Y. Lu, X. Zhao, and J. Wang, "ClinicalRAG: Enhancing Clinical Decision Support through Heterogeneous Knowledge Retrieval," in *Proc. 1st Workshop on Towards Knowledgeable Language Models (KnowLLM 2024)*, pp. 64-68, 2024, doi: 10.18653/v1/2024.knowllm-1.6.

[4] K. Zuo, Y. Jiang, F. Mo, and P. Lio, "KG4Diagnosis: A Hierarchical Multi-Agent LLM Framework with Knowledge Graph Enhancement for Medical Diagnosis," in *Proc. First AAAI Bridge Program on AI for Medicine and Healthcare*, PMLR, vol. 281, pp. 195-204, 2025.

[5] T. Abdullahi, L. Mercurio, R. Singh, and C. Eickhoff, "Retrieval-Based Diagnostic Decision Support: Mixed Methods Study," *JMIR Medical Informatics*, vol. 12, e50209, 2024, doi: 10.2196/50209.

[6] S. Liu, A. B. McCoy, and A. Wright, "Improving large language model applications in biomedicine with retrieval-augmented generation: a systematic review, meta-analysis, and clinical development guidelines," *Journal of the American Medical Informatics Association*, vol. 32, no. 4, pp. 605-615, 2025, doi: 10.1093/jamia/ocaf008.

[7] S. Delourme, A. Redjdal, J. Bouaud, and B. Seroussi, "Measured Performance and Healthcare Professional Perception of Large Language Models Used as Clinical Decision Support Systems: A Scoping Review," *Studies in Health Technology and Informatics*, vol. 316, pp. 841-845, 2024, doi: 10.3233/SHTI240543.

[8] C. Wang, J. Ong, C. Wang, H. Ong, R. Cheng, and D. Ong, "Potential for GPT Technology to Optimize Future Clinical Decision-Making Using Retrieval-Augmented Generation," *Annals of Biomedical Engineering*, vol. 52, no. 5, pp. 1115-1118, 2024, doi: 10.1007/s10439-023-03327-6.

[9] S. Kresevic, M. Giuffrè, M. Ajcevic, A. Accardo, L. S. Crocè, and D. L. Shung, "Optimization of hepatological clinical guidelines interpretation by large language models: a retrieval augmented generation-based framework," *npj Digital Medicine*, vol. 7, art. no. 102, 2024, doi: 10.1038/s41746-024-01091-y.

[10] W. Li, R. Yan, X. Zhang, L. Chen, H. Zhu, J. Zhao, J. Li, M. Li, W. Cao, Z. Jiang, W. Wei, K. Zhang, and S. K. Zhou, "MACD: Multi-Agent Clinical Diagnosis with Self-Learned Knowledge for LLM," arXiv preprint arXiv:2509.20067, 2025.

[11] A. J. Thirunavukarasu, D. S. J. Ting, K. Elangovan, L. Gutierrez, T. F. Tan, and D. S. W. Ting, "Large language models in medicine," *Nature Medicine*, vol. 29, no. 8, pp. 1930-1940, 2023, doi: 10.1038/s41591-023-02448-8.

[12] A. Patel, Y. Khand, S. K. Vallamchetla, P. Li, C. Tao, and J. Cheung, "Fine-Tuning, Retrieval-Augmented Generation, and Hybrid Adaptation of Language Models for Clinical Decision-Making in Health Care: Systematic Review," *Journal of Medical Internet Research*, vol. 28, e104092, 2026, doi: 10.2196/104092.

APPENDIX C : PLAGIARISM ANALYSIS

Date: 15-10-2026

Plagiarism Scan Report

	0% Exact Match		Words
0% Plagiarism		100% Unique	Characters
			Sentences
	0% Partial Match		Paragraphs
			Read Time 105 minute(s)
			Speak Time 140 minute(s)

Content Checked For Plagiarism

CHAPTER 01
INTRODUCTION
1.1 PROJECT OVERVIEW AND CORE CONCEPT :
1.2 MOTIVATION: MAKING CLINICAL AI TRACEABLE AND AUDITABLE :
1.3 PROBLEM STATEMENT: LIMITATIONS OF SINGLE-MODEL CLINICAL DECISION SUPPORT :
1.4 PROJECT AIM, OBJECTIVES, AND EXPECTED OUTCOMES :
1.5 PROJECT SCOPE :

Matched Source

Congratulations!

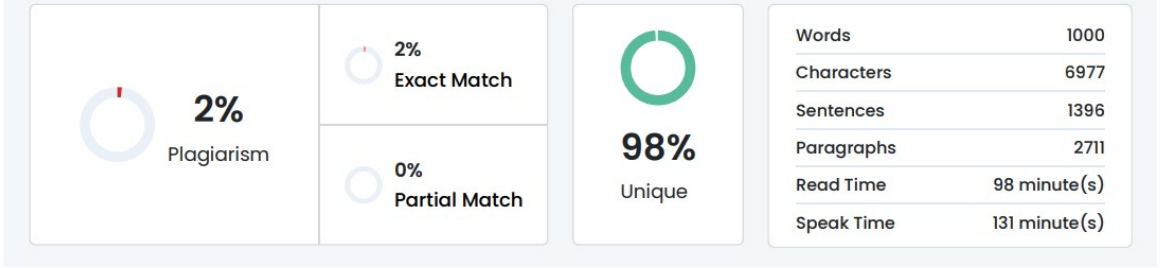
No Plagiarism Found

Check By: Dupli Checker

DupliChecker

Date: 15-10-2026

Plagiarism Scan Report



Content Checked For Plagiarism

CHAPTER 02

LITERATURE REVIEW

2.1 OVERVIEW OF AI IN CLINICAL DECISION SUPPORT :

2.2 CRITICAL REVIEW OF EXISTING SYSTEMS AND IDENTIFIED GAPS :

2.2.2 CONCEPTUAL DIAGRAM OF EXISTING CDSS APPROACHES :

Matched Source

Similarity 2%

Title:The Role of AI in Clinical Decision Support Systems (CDSS)

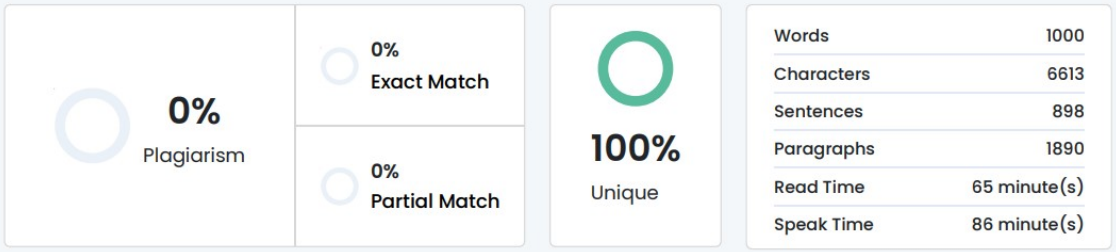
Nov 2, 2025 — AI-driven Clinical Decision Support Systems provide the much-needed intelligence layer to assist doctors in diagnosis, risk assessment, and ...Read more View Source

Check By: Dupli Checker

DupliChecker

Date: 15-10-2026

Plagiarism Scan Report



Content Checked For Plagiarism

CHAPTER 03
SOFTWARE & HARDWARE REQUIREMENT SPECIFICATION
3.1 INTRODUCTION TO SYSTEM REQUIREMENTS :
3.1.1 PROJECT SCOPE :
3.1.2 USER CLASSES AND CHARACTERISTICS :
3.2 FUNCTIONAL REQUIREMENTS (FRS) :

Matched Source

Congratulations!

No Plagiarism Found

Check By: Dupli Checker

APPENDIX D : PUBLICATION

